

Accepted Manuscript

International Journal of Information Technology & Decision Making

Article Title: An Improved Particle Swarm Optimization Algorithm with Adaptive Inertia Weights

Author(s): Mi Li, Huan Chen, Xiaodong Wang, Ning Zhong, Shengfu Lu

DOI: 10.1142/S0219622019500147

Received: 07 January 2016

Accepted: 10 February 2019

To be cited as: Mi Li *et al.*, An Improved Particle Swarm Optimization Algorithm with Adaptive Inertia Weights, *International Journal of Information Technology & Decision Making*, doi: 10.1142/S0219622019500147

Link to final version: <https://doi.org/10.1142/S0219622019500147>

This is an unedited version of the accepted manuscript scheduled for publication. It has been uploaded in advance for the benefit of our customers. The manuscript will be copyedited, typeset and proofread before it is released in the final form. As a result, the published copy may differ from the unedited version. Readers should obtain the final version from the above link when it is published. The authors are responsible for the content of this Accepted Article.

2018 年 12 月 13 日 19:31 WSPC/ws-ijitdm pso

International Journal of Information Technology & Decision Making
© World Scientific Publishing Company

AN IMPROVED PARTICLE SWARM OPTIMIZATION ALGORITHM WITH ADAPTIVE INERTIA WEIGHTS

Received Day Month Year

Revised Day Month Year

The particle swarm optimization (PSO) algorithm is simple to implement and converges quickly, but it easily falls into a local optimum; on the one hand, it lacks the ability to balance global exploration and local exploitation of the population, and on the other hand, the population lacks diversity. To solve these problems, this paper proposes an improved adaptive inertia weight particle swarm optimization algorithm (AIWPSO). The AIWPSO algorithm includes two strategies: 1) An inertia weight adjustment method based on the optimal fitness value of individual particles is proposed, so that different particles have different inertia weights. This method increases the diversity of inertia weights and is conducive to balancing the capabilities of global exploration and local exploitation. 2) A mutation threshold is used to determine which particles need to be mutated. This method compensates for the inaccuracy of random mutation, effectively increasing the diversity of the population. To evaluate the performance of the proposed AIWPSO algorithm, benchmark functions are used for testing. The results show that AIWPSO achieves satisfactory results compared with those of other PSO algorithms. This outcome shows that the AIWPSO algorithm is conducive to balancing the abilities of the global exploration and local exploitation of the population, while increasing the diversity of the population, thereby significantly improving the optimization ability of the PSO algorithm.

Keywords: particle swarm optimization; adaptive inertia weight; mutation threshold; mutation; diversity.

1. Introduction

Particle swarm optimization (PSO) is a stochastic evolutionary algorithm motivated by the simulation of social behavior. It was originally introduced by Kennedy

and Eberhart¹. The algorithm updates every individual from a population by applying types of operators based on the fitness information obtained from the environments. At present, many other optimization algorithms, such as genetic algorithm, ant colony algorithm and artificial neural network, have also been extensively studied. These optimization algorithms and particle swarm optimization algorithm were used to solve many real world optimization problems. For example, Valle et al.² provided a comprehensive survey on the power system applications that profited from the powerful nature of PSO as an optimization technique. The modified evolution direction operator (MEDO) was used into the traditional particle swarm optimization algorithm (PSO) to bring IPSO method³ for training the three-layer neural network and classifying the EEG signals. This method achieved a good effect on the classification of mental tasks from EEG data. Wai et al.⁴ designed a real-time PID control method for the suspension balance and propulsion positioning of the magnetic levitation transport system through PSO gain selection. The system had excellent control performance, and had no chattering phenomenon in SM control. It's robustness to uncertainty was better than fixed gain PSO-PID control. Using the linear decreasing inertia weight PSO algorithm to determine the free parameters of support vector machine (SVM) could construct a PSO-SVM classification model⁵ to predict rainfall-runoff combined with the ensemble empirical model decomposition (EEMD). This method significantly improved the accuracy of rainfall-runoff prediction at the research station. Chau et al.⁶ proposed a split-step particle swarm optimization algorithm that divided the process of training perceptron into two steps: firstly using a random PSO for global search; secondly using the Levenberg-Marquardt (LM) algorithm to fine tune the local search. This method was applied to the multi-layer perceptron of real-time water level prediction. The results showed that the algorithm could avoid the shortcomings of PSO global search taking longer and the LM algorithm being easy to fall into local minima, thus obtained higher precision in a shorter time. A semi-supervised local linear embedding method (semi-SLLE) emerged based on a new distance metric method⁷. It was applied to the feature extraction and processing of plant leaf recognition to classify the plants by leaf image combined with K-NN. It was found that the algorithm performed well on the leaf image data of the manifold structure. Gholami et al.⁸ combined the artificial neural network (ANN) and the tree chronology (tree ring) to simulate the groundwa-

ter level fluctuation: the tree wheel diameter and precipitation were acted as input parameters, and the groundwater level was acted as the output parameters. The results showed that the model achieved high precision and efficiency. A bit-flipping mutation operator was added into the binary-coded particle swarm optimization algorithm to generate a binary-coded variant of the particle swarm optimization algorithm⁹ for optimizing the filter parameters and the extreme learning machine model structure in river prediction. The result showed that this algorithm greatly reduced the calculation time. Fotovatikhah et al.¹⁰ evaluated the advantages and disadvantages of single computational intelligence (CI) methods and mixed computational intelligence (CI) methods these that were applied to flood management systems (FMSS) and the accuracy of error rates. The hybrid approach was a potential method to improve the accuracy and the lead time of flood and debris flow predictions. A standard particle swarm optimization algorithm was combined with an integrated method based on fuzzy support vector data description (SVDD) and a KNN to construct a hybrid algorithm (referred to as PSO-FSVDK)¹¹. The study of the performance of the algorithm in personal credit rating showed that the hybrid algorithm obtained higher classification accuracy than some existing credit scoring methods. In real life, these optimization algorithms had a wide application prospect, such as financial time series forecasting problem¹², pairwise comparison matrix research in multiple criteria decision making¹³, cardinal and ordinal inconsistencies problem in pairwise comparison matrix¹⁴ and real estate investment program evaluation problems¹⁵. However, PSO was simpler in implementation and was more excellent than other evolutionary algorithms¹⁶.

In PSO, every potential solution to an optimization problem is considered as a particle whose state is described by position and velocity vectors: the former represents feasible solutions, and the latter represents the direction of motion in the search space. Suppose that m particles are used to search the solution. The position of the i^{th} particle is expressed as $x_i = (x_{i1}, x_{i2}, \dots, x_{iD})$, where $x_{id} \in [pop_{min}, pop_{max}]$, $d \in \{1, 2, \dots, D\}$, pop_{min} and pop_{max} are the lower and upper bounds of the position of the particle. The velocity of the i^{th} particle is expressed as: $v_i = (v_{i1}, v_{i2}, \dots, v_{iD})$, where $v_{id} \in [v_{min}, v_{max}]$, $d \in \{1, 2, \dots, D\}$, and v_{min} and v_{max} are the lower and upper bounds, respectively, of the velocity of the particle. The positions x_i and velocities v_i of the particles are initialized randomly, and the

position and velocity of each particle change according to the position and velocity of the particle itself and its neighbors¹. The updated equations are given below:

$$v_{ij}(t+1) = v_{ij}(t) + c_1 rand_1(pbest_{ij}(t) - x_{ij}(t)) + c_2 rand_2(gbest_j(t) - x_{ij}(t)) \quad (1)$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1) \quad (2)$$

where t is the current number of iterations, v_{ij} is the j^{th} velocity coordinate of the i^{th} particle, x_{ij} is the j^{th} position coordinate of the i^{th} particle, $pbest_{ij}$ is the j^{th} dimension of the optimal position of the i^{th} particle, and $gbest_j$ is the j^{th} dimension of the optimal position of all particles of the swarm; c_1 and c_2 are the acceleration parameters, and $rand_1$ and $rand_2$ separately represent two randomly generated numbers in the range of $[0, 1]$.

The PSO algorithm has been extensively studied, and many improved methods have been proposed. These include: 1) improving the acceleration factor and inertia weight; 2) changing the topology of the particles to increase the diversity of the population, such as ring topology or square topology; 3) hybrid algorithms, such as Gaussian mutation, self-organized criticality control¹⁷, using a mutation strategy of genetic algorithms and fuzzy algorithms^{18,19} (for example, Melin used fuzzy logic methods to improve the convergence speed and population diversity^{20,21}); and lastly, 4) embedding mathematical and physical techniques, such as quantum-behaved particle swarm algorithms²².

One of the keys to the success of the PSO algorithm is the ability of the algorithm to balance global exploration and local exploitation throughout the entire operation process²³. Shi and Eberhart²⁴ first proposed a PSO based on inertia weight w to achieve a balance between exploration and exploitation, in which the inertia weight was a constant between $(0,1]$, and updated the velocity of each particle according to the following equation²⁴:

$$v_{ij}(t+1) = wv_{ij}(t) + c_1 rand_1(pbest_{ij}(t) - x_{ij}(t)) + c_2 rand_2(gbest_j(t) - x_{ij}(t)) \quad (3)$$

Equation (3) indicates that the balance of global exploration ability and local exploitation ability of the algorithm can be affected by adjusting the inertial weight w . Here, the inertial weights of all particles correspond to the same constant value throughout the iteration. When all other parameters are the same, the larger inertia weight is beneficial to global exploration, and the smaller inertia weight is beneficial

to local exploitation²⁵. Therefore, when the inertial weight is a fixed constant value, the global exploration and local exploitation of the population cannot be well considered.

Later, Shi and Eberhart²⁶ proposed the random inertia weight adjustment method. In this method, the inertia weight is a random number. Eberhart²⁷ proposed a linear time-varying inertial weight adjustment method. In this method, the inertia weight decreases linearly with the number of iterations. Chatterjee and Siarry²⁸ proposed a nonlinear time-varying inertial weight adjustment method. In this method, the inertia weight decreases nonlinearly with the number of iterations. There are two disadvantages to these inertial weight improvement methods. One is that it does not consider the particle's current search conditions; the other is the lack of adaptability in particle search. Therefore, adaptive inertia weight is widely studied^{29,30}. For example, Zhang et al.³¹ proposed a Cauchy adaptive inertia weight based on the Bayesian criterion approach. In this method, the inertia weight is adjusted according to the position of past particles using Bayesian technology. Hu and Yen³² presented an adaptive inertia weight with parallel cell cooperation mechanisms for PSO algorithms. In this method, the inertia weight is adjusted according to the distribution entropy of the particles. Arumugam and Rao³⁰ proposed an adaptive inertia weight adjustment method that monitors search conditions based on particle fitness. In this method, the inertia weight is adjusted according to the average fitness value of particles. In these adaptive inertial weight improvement methods, the particle search has a certain degree of self-adaptability, but the inertial weights of all particles are the same in each iteration, and the particles cannot completely search in the search space according to their own search conditions. Therefore, these improved methods still do not balance the exploration and exploitation of the population well. Aiming at this problem, we propose an adaptive inertial weight adjustment method. This method is based on the optimal fitness value of particle.

In the adaptive inertial weight adjustment method proposed in this paper, the random function $\text{rand}()$ is used to make the inertial weight of the particles different in each iteration, and the inertial weight of each particle is determined by the ratio of the global optimal fitness value of all particles to the optimal fitness value of the particle. This method enables different particles to obtain different

inertial weight values, and enables the particles to conduct completely adaptive search in the search space, which is helpful to balance the global exploration and local exploitation ability of the population.

The improved adaptive inertia weight is beneficial for balancing the abilities of global exploration and local exploitation of the population, while another key to the PSO algorithm falling into a local optimum is the lack of diversity of the population³³. Usually, the mutation of particles is used to increase the diversity of the population. The ideal mutation would be for particles closer to the optimal solution to mutate and move to other areas for exploration, while particles farther away from the optimal solution will continue searching at their current location. But in previous studies^{33,34}, mutant particles were randomly selected, and could not accurately pick which ones need to be mutated. Since the positions of the particles in each iteration are uncertain, it is impossible to simply set a fixed value or a random value to determine whether the distance between the particle and the optimal solution is far or near. To fully consider the current position of the particle, we use the average Euclidean distance from the position of the successful particle to the globally optimal position as the threshold. The threshold is a dynamic value, so that the particle can judge whether it needs to mutate according to the Euclidean distance from its position to the global optimal position: mutate when it is less than the threshold, otherwise it will not change. Therefore, the mutation threshold is used to determine which particles need to be mutated, making up for the limitations of random mutation and thus effectively increasing the diversity of the population.

Based on the improved adaptive inertia weight and the mutation threshold, we propose an improved adaptive inertia weight particle swarm optimization algorithm (AIWPSO). To evaluate the effectiveness of the proposed AIWPSO algorithm, we use 18 benchmark functions to test its performance in finding optimal solutions. The experiments demonstrate that the proposed algorithm achieves satisfactory optimization performance.

The remainder of this paper is organized as follows. Section 2 briefly reviews the research on inertia weights (w) in PSO. The proposed adaptive particle swarm optimization algorithm is described in Section 3. Section 4 gives the experimental results and proves the effectiveness of the proposed AIWPSO algorithm by comparing it with other methods. Finally, Section 5 summarizes this study.

2. Related Work

This section first gives a brief review of the adjustment methods of inertia weight. Then, the latest and most typical methods are given. Because there are many ways to improve the inertia weight w , we divide them into two categories, the constant strategy and the time-varying strategy, which determine the value of the inertia weight according to the number of iterations.

2.1. Constant Value Strategy

The constant value strategy is such that the inertia weight is a fixed value during the entire search process.

Shi and Eberhart use a series of constant values to test the performance of the algorithm and show that by using large values of w , i.e., $w > 1.2$, PSO only performs a weak exploration, and with low values of this parameter, i.e., $w < 0.8$, PSO tends to fall into local optima ²⁴.

2.2. Time-varying Strategies

The PSO using the time-varying strategies mean that the value of the inertia weight is determined according to the number of iterations. These methods can be linear or nonlinear, and increased or decreased.

2.2.1. Random Strategy

The random inertia weight method is proposed by Shi and Eberhart ²⁶. The inertia weight is randomly determined according to the following equation:

$$w(k) = 0.5 + \frac{rand}{2} \quad (4)$$

where $rand$ represents randomly generated numbers in the range of $[0, 1]$.

2.2.2. Linear Time-varying Strategy

A strategy with a linearly decreasing inertia weight is introduced by Eberhart ²⁷. It improves the performance of PSO by varying the inertial weight. In this method, the inertia weight linearly decreases from the initial value w_{max} to the final value w_{min} based on the following equation:

$$w(k) = w_{min} + (w_{max} - w_{min}) \frac{(T_{max} - k)}{T_{max}} \quad (5)$$

where k is the current iteration number of the algorithm and T_{max} is the maximum number of iterations allowed by PSO. w_{max} and w_{min} are the maximum and minimum of the inertia weight, respectively.

2.2.3. Nonlinear Time-varying Strategy

Chatterjee and Siarry²⁸ propose a nonlinearly decreasing variant of the inertia weight, and w is determined according to the following equation:

$$w(k) = w_{min} + (w_{max} - w_{min}) \frac{(T_{max} - k)^n}{(T_{max})^n} \quad (6)$$

where n is a nonlinear adjustment index.

Feng³⁵ uses a chaotic model of the inertia weight, where the chaotic condition is added to the linearly decreasing inertia weight. The proposed w is determined according to the following equation:

$$w(k) = w_{min} z(k) + (w_{max} - w_{min}) \frac{(T_{max} - k)}{T_{max}} \quad (7)$$

where

$$z(k) = \begin{cases} rand & k = 1 \\ 4z(k-1)(1 - z(k-1)) & k > 1 \end{cases} \quad (8)$$

Fan³⁶ presents a method in which w is exponentially and nonlinearly decreasing as follows:

$$w = (2/k)^{0.3} \quad (9)$$

Jiao³⁷ proposes a nonlinearly increasing inertia weight method in which w increases nonlinearly with the number of iterations; the equation is as follows:

$$w(k) = w_{initial} \times u^k \quad (10)$$

where $w_{initial}$ is the initial inertia weight which is in the range of $[0, 1]$, and u is a constant value in the range of $[1.0001, 1.005]$. In the experiments conducted, u is usually set to 1.0002, and the algorithm exhibits good performance.

In these methods, w is varied according to the number of iterations; hence, the strategy is a time-varying inertia weight strategy. The disadvantage of this strategy

is that w cannot be adjusted according to the state of the particles; it can only increase or decrease, as there is no feedback mechanism.

2.2.4. Adaptive Strategy

The adaptive inertia weight strategy adjusts the inertia weight by using one or more feedback parameters by monitoring the search situation of the algorithm. Arumugam and Rao determine the inertia weight of each iteration using the ratio of the global optimal fitness to the average value of the local optimal fitness of the particles, in which w is determined by the following equation ³⁰:

$$w_i(k) = 1.1 - \frac{fitness(g_{best})(k)}{fitness(p_{best})_{average}(k)} \quad (11)$$

In the adaptive inertia weight strategy proposed by Panigrahi et al., the inertia weight is assigned to different particles according to the rank of the particles. The equation is as follows ³⁸:

$$w_i = w_{min} + (w_{max} - w_{min}) \frac{Rank_i}{Total_{population}} \quad (12)$$

where $Rank_i$ is the sorting position of the particle in each iteration, and $Total_{population}$ is the number of particles.

In addition to the above inertia weight methods, there are many inertia weight methods proposed. A summary of various inertia weight strategies is given in Table 1. The inertia weight methods, adjustment strategies and feedback parameters are listed in the table. In most inertial weight methods, the inertia weight of particles has the same value in each iteration, which is not conducive to having a particle search in the search space according to its own conditions. To solve this problem, we increase the diversity of the inertia weights based on the individual optimal fitness of the particles and make the particles search in the search space according to each individual's own situation. The proposed PSO algorithm is described in Section 3.

3. The Proposed AIWPSO Algorithm

Although PSO is simple to implement and converges quickly, it is easily trapped in local optima; on the one hand, it lacks the ability to balance global exploration and local exploitation of the population, and on the other hand, the population lacks diversity. To improve the ability of the PSO to balance global exploration and local

Table 1. Summary of various modification mechanisms of inertia weight.

Inertia weight method	Reference	Adjustment strategy	Feedback parameter
$w = c$	24	-	-
$w(k) = 0.5 + \frac{rand}{2}$	26	Random	-
$w(k) = w_{min} + (w_{max} - w_{min}) \frac{(T_{max} - k)}{T_{max}}$	27	Linear time-varying	-
$w(k) = w_{max} + (w_{min} - w_{max}) \frac{(T_{max} - k)}{T_{max}}$	28	Linear time-varying	-
$w(k) = w_{min} + (w_{max} - w_{min}) \frac{(T_{max} - k)^n}{(T_{max})^n}$	28	Nonlinear time-varying	-
$w(k) = w_{initial} \times u^k$	37	Nonlinear time-varying	-
$w(k) = w_{min}z(k) + (w_{max} - w_{min}) \frac{(T_{max} - k)}{T_{max}}$	35	Linear time-varying with random changes	-
$w(k) = \frac{(1 - \frac{k}{T_{max}})}{(1 - s(\frac{k}{T_{max}}))}$	39	Nonlinear time-varying	-
$w = (2/k)^{0.3}$	36	Nonlinear time-varying	-
Cauchy criterion	16	Adaptive	-
Fuzzy rule	16	Adaptive	Fitness
$w(k) = 1.1 - \frac{g_{best}(k)}{(p_{best})_{average}(k)}$	30	Adaptive	Fitness
$w_i = w_{min} + (w_{max} - w_{min}) \frac{Rank_i}{Total population}$	38	Adaptive	Particle rank
$w_{ij} = 1 - \alpha(\frac{1}{1 + e^{ISA_{ij}}})$ $ISA_{ij} = \frac{ x_{ij} - p_{ij} }{(p_{ij} - p_{gj} + \epsilon)}$	40	Adaptive	Distance

exploitation, an inertia weight adjustment method based on the optimal fitness value of individual particles is proposed so that different particles have different inertia weights, increasing the diversity of inertia weights. Therefore, the particles can search in the search space according to their own conditions and is conducive to balancing the capabilities of global exploration and local exploitation. In addition, to increase the diversity of the population, we use the mutation threshold to determine which particles need to be mutated. This method compensates for the limitations of random mutation, which can effectively increase the diversity of the population.

3.1. Diversity Enhancement Mechanism of Inertia Weight

Arumugam's adaptive inertia weight model is based on the ratio of the global optimal fitness to the average of the particles' personal optimal fitness³⁰. The problem with this method is that all particles' inertia weights have the same value in the process. Using the same weight to affect the speed of the next generation, assuming that other parameters are the same, means that when w is smaller, the speed change is small and all particles tend to local exploitation. When w is bigger,

the particle's velocity changes more and all particles tend to global exploration. Therefore, this model cannot balance well the abilities of global exploration and local exploitation of the population.

To better balance the abilities of global exploration and local exploitation of the population, an inertia weight adjustment method based on the optimal fitness of individual particles is proposed. The improved adjustment method is shown in the following formula (13), and the particle personal optimal fitness $pbest_i(k)$ is used as a feedback parameter. In each iteration, each particle will get a w related to the current search condition (fitness) so that particles with different weights affect the speed of the next generation. For particles with smaller w , the velocity changes less and tends to local exploitation. For particles with larger w , the velocity changes more, which is often a global exploration. This improved adjustment method implements the division of labor and solves the problems in the Arumugam's model³⁰.

The improved adaptive inertia weight equation based on personal optimal fitness is as follows:

$$w_i(k) = 0.5rand + \frac{fitness(g_{best})(k)}{fitness(p_{best})_i(k)} \quad (13)$$

where $w_i(k)$ is the inertia weight of the i^{th} particle at the k^{th} iteration, $fitness(g_{best}(k))$ is the global optimal fitness at k^{th} iteration, $fitness(p_{best})_i$ is the optimal fitness of the i^{th} particle, $\frac{fitness(g_{best})(k)}{fitness(p_{best})_i(k)}$ is in the range of $[0, 1]$, w is in the range of $[0.5, 1.5]$.

3.2. Mutation Threshold Strategy

Mutation can increase the diversity of the population because the mutated particles can go to other areas of the search space to explore. The choice of mutated particles is key to the algorithm. In previous studies, mutated particles are randomly selected at each iteration^{33,34}. This method cannot accurately choose the particles that need to be mutated, thus reducing the convergence speed of the algorithm. In general, we hope that particles farther away from the optimal solution can continue to search at the current position, and particles closer to the optimal solution will mutate so that particles can explore in other regions, which can prevent the population from falling into the local optimum prematurely. Because most of the particles

are moving to the current globally optimal position during the iterative process, the diversity of the population is bound to be greatly reduced. Therefore, we hope to increase the diversity of the population by mutation of the particles near the optimal solution. Particles that have successfully searched are particles that are slowly moving to the present global optimum position. Because whether the distance between the particle and the optimal solution is far or near cannot be decided without having a threshold to judge distance with, and the position of the particle in each iteration is an undefined value, a fixed value cannot be simply used as a threshold. To fully integrate the positions of the particles, we use the $dist_{average}$, which is the average value of the Euclidean distance from the location of the successful particle to the global optimal position as the threshold; $dist_{average}$ is a dynamic value, which helps the particles to mutate according to their own position, thus increasing the diversity of the population. If $dist_i > dist_{average}$, these particles continue searching in the current position. If $dist_i \leq dist_{average}$, these particles choose to mutate and improve the global exploration ability. This method preserves the ability of local searching while taking into account the ability of global exploration.

In the minimization problem, the successful particle in the k^{th} iteration is defined as ¹⁶:

$$SS(i, k) = \begin{cases} 1 & \text{if } fit(pbest_i^k) < fit(pbest_i^{k-1}) \\ 0 & \text{if } fit(pbest_i^k) = fit(pbest_i^{k-1}) \end{cases} \quad (14)$$

where $SS(i, k) = 1$ indicates that the particle i searches succeeded, $SS(i, k) = 0$ indicates that the particle i searches failed, and $fitness(pbest_i^k)$ is the optimal fitness for the current position found by particle i at iteration k . The number of successful particles m is determined according to the following equation:

$$m = \sum_{i=1}^{popsize} SS(i, k) \quad (15)$$

The percentage of successful searches is determined as:

$$pss(k) = \frac{m}{popsize} \times 100\% \quad (16)$$

where $popsize$ is the number of population, m is the number of successful particles.

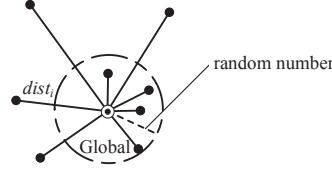


Fig. 1. Random number as a threshold to select the required mutation particles.

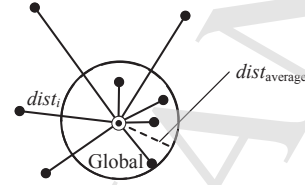
Fig. 2. $dist_{average}$ as a threshold to select the required mutation particles.

Fig.1 is a schematic diagram that does not determine which particles need to be mutated. Fig.2 provides a schematic diagram of the determined mutation particles. The black dots represent the particles, and the circled dot represents the globally optimal position of the particles in the current iteration. The Euclidean distance from the position of the i^{th} particle to the position of the current global optimal particle is calculated as follows:

$$dist_i = \left(\sum_{j=1}^D (gbest_j - x_{ij})^2 \right)^{0.5} \quad (17)$$

where $gbest_j$ is the j^{th} dimension of the current global optimal position, x_{ij} is the j^{th} position coordinate of the i^{th} particle, and D is the search space's dimension. The formula for calculating the mean value of the Euclidean distance from the position of the successful particle to the current global optimal position is as follows:

$$dist_{average} = \frac{1}{m} \sum_{i=1}^m dist_i \quad (18)$$

where m is the number of successful particles.

$dist_{average}$ is the threshold for identifying particles to be mutated. The mutation condition is defined as follows:

$$mut_i = \begin{cases} 1 & \text{if } dist_i \leq dist_{average} \\ 0 & \text{if } dist_i > dist_{average} \end{cases} \quad (19)$$

where $mut_i = 1$ indicates that the i^{th} particle satisfies the mutation condition, and $mut_i = 0$ indicates it does not meet.

The mutation of the i^{th} particle can be obtained by the following equation ⁴¹:

$$pop(i) = (pop_{max} - pop_{min}) \times rand + pop_{min} \quad (20)$$

where pop_{max} and pop_{min} are the upper and lower bounds of the position of the particle, and $rand$ represents a randomly generated number in the range of $[0, 1]$.

3.3. The Proposed Approach

The proposed algorithm adopts two strategies: the inertia weight diversity enhancement mechanism and the mutation threshold strategy. The former aims at balancing the abilities of global exploration and local exploitation of the population, while the latter focuses on increasing the diversity of the population to prevent the algorithm from falling into a local optimum.

The pseudo-code of the proposed AIWPSO is shown in Algorithm 3.1, and the pseudo-codes of the other algorithms with improved inertia weights (constant inertia weight PSO (CPSO) ²⁴, random inertia weight PSO (RPSO) ²⁶, linear decreasing inertia weight (LDPSO) ^{24,42}, nonlinear decreasing inertia weight PSO (NLDPSO) ²⁸ and adaptive inertia weight PSO (APSO) ³⁰) being compared are shown in Algorithms 3.2-3.6.

4. Experimental Results

The experimental platform is a Lenovo M490 PC, 32-bit Windows 7 operating system, third-generation Intel Core i5 processor; the frequency of the CPU is 2.50 GHz, the memory is 4GB, and the software version is MATLAB R2013b, which is simpler and easier to implement than other software.

Algorithm 3.1 The proposed AIWPSO algorithm.

```

1: Parameters initialization;
2: Uniformly randomly initialize each particle in the swarm;
3: Calculate the fitness of  $x_i$ ,  $pbest_i$  and  $gbest$ ;
4: for  $k=1$  to  $Tmax$  do
5:   for  $i=1$  to population size do
6:     Calculate the inertia weight according to  $w_i(k) = 0.5rand + \frac{fitness(gbest)(k)}{fitness(pbest_i)(k)}$ ;
7:     Calculate the velocity of particle  $x_i$  according to Eq. (3);
8:     Update the position of particle  $x_i$  according to Eq. (2);
9:     Calculate the fitness of  $x_i$ , and update  $pbest_i$  and  $gbest$ ;
10:  end for
11:   $m = 0$ ;
12:  for  $i=1$  to population size do
13:    if  $pbest_i(k) < pbest_i(k-1)$  then
14:       $m = m + 1$ ;
15:      Calculate the Euclidean distance between the particle's position and the current
        global optimal position according to  $dist_i = (\sum_{j=1}^D (gbest_j - x_{ij})^2)^{0.5}$ ;
16:    end if
17:  end for
18:  Calculate the threshold according to  $dist_{average} = \frac{1}{m} \sum_{i=1}^m dist_i$ ;
19:  for  $i=1$  to population size do
20:    if  $dist_i \leq dist_{average}$  then
21:      The particle mutates;
22:    end if
23:  end for
24:   $k++$ ;
25: end for

```

4.1. Test Functions

In order to benchmark the performance of AIWPSO, we conduct simulations on 18 different benchmark functions. These benchmark functions are all the base functions from the competition on real-parameter single objective optimization problems at Congress on Evolutionary Computation2010(CEC2010)⁴³ and Congress on Evolutionary Computation2014(CEC2014)⁴⁴, and they are listed in Table 2 (the formula of f13-f18 is very complex, so there is no listing here, but its detailed description can be found in CEC2014⁴⁴). All of them are minimization problems, which have different characteristics and can fully investigate the optimization ability of the

Algorithm 3.2 CPSO algorithm.

```

1: Parameters initialization;
2: Uniformly randomly initialize each particle in the swarm;
3:  $w = 1$ ;
4: Calculate the fitness of  $x_i$ ,  $pbest_i$  and  $gbest$ ;
5: for  $k=1$  to Tmax do
6:   for  $i=1$  to population size do
7:     Update the velocity of particle  $x_i$  according to Eq. (3);
8:     Update the position of particle  $x_i$  according to Eq. (2);
9:     Calculate the fitness of  $x_i$ ;
10:    Update  $pbest_i$  and  $gbest$ ;
11:   end for
12:    $k++$ ;
13: end for

```

Algorithm 3.3 RPSO algorithm.

```

1: Parameters initialization;
2: Uniformly randomly initialize each particle in the swarm;
3: Calculate the fitness of  $x_i$ ;
4: Update  $pbest$  and  $gbest$ ;
5: for  $k=1$  to Tmax do
6:   for  $i=1$  to population size do
7:     Calculate the inertia weight according to  $w(k) = 0.5 + \frac{rand}{2}$ ;
8:     Calculate the velocity of particle  $x_i$  according to Eq. (3);
9:     Update the position of particle  $x_i$  according to Eq. (2);
10:    Calculate the fitness of  $x_i$ ;
11:    Update  $pbest_i$  and  $gbest$ ;
12:   end for
13:    $k++$ ;
14: end for

```

algorithm.

4.2. Parameter Settings

To verify the effectiveness and efficiency of our proposed algorithm, we compare AIWPSO with five other PSO algorithms with improved inertia weights (CPSO ²⁴, RPSO ²⁶, LDPSO ^{24,42}, NLDPSO ²⁸ and APSO ³⁰). In these tests, all PSO algo-

Algorithm 3.4 LDPSO algorithm.

```

1: Parameters initialization;
2: Uniformly randomly initialize each particle in the swarm;
3: Calculate the fitness of  $x_i$ ;
4: Update  $pbest$  and  $gbest$ ;
5: for  $k=1$  to  $Tmax$  do
6:   for  $i=1$  to population size do
7:     Calculate the inertia weight according to  $w(k) = w_{min} + (w_{max} - w_{min}) \frac{(T_{max} - k)}{T_{max}}$ ;
8:     Calculate the velocity of particle  $x_i$  according to Eq. (3);
9:     Update the position of particle  $x_i$  according to Eq. (2);
10:    Calculate the fitness of  $x_i$ ;
11:    Update  $pbest_i$  and  $gbest$ ;
12:   end for
13:    $k++$ ;
14: end for

```

Algorithm 3.5 NLDPSO algorithm.

```

1: Parameters initialization;
2: Uniformly randomly initialize each particle in the swarm;
3: Calculate the fitness of  $x_i$ ;
4: Update  $pbest$  and  $gbest$ ;
5: for  $k=1$  to  $Tmax$  do
6:   for  $i=1$  to population size do
7:     Calculate the inertia weight according to  $w(k) = w_{min} + (w_{max} - w_{min}) \frac{(T_{max} - k)^2}{(T_{max})^2}$ ;
8:     Calculate the velocity of particle  $x_i$  according to Eq. (3);
9:     Update the position of particle  $x_i$  according to Eq. (2);
10:    Calculate the fitness of  $x_i$ ;
11:    Update  $pbest_i$  and  $gbest$ ;
12:   end for
13:    $k++$ ;
14: end for

```

rithms are tested using the same population size, 20. Each algorithm performs 30 independent trials on every test function with the same maximum number of iterations, $Tmax = 20000$. The parameter settings of the six PSO algorithms are based on the corresponding references, as shown in Table 3. In addition, these parameter settings are good choices in the relevant literature, and have achieved good

Algorithm 3.6 APSO algorithm.

-
- 1: Parameters initialization;
 - 2: Uniformly randomly initialize each particle in the swarm;
 - 3: Calculate the fitness of x_i ;
 - 4: Update $pbest$ and $gbest$;
 - 5: **for** $k=1$ to $Tmax$ **do**
 - 6: **for** $i=1$ to population size **do**
 - 7: Calculate the inertia weight according to $w(k) = 1.1 - \frac{fit(g_{best}(k))}{fit((p_{best})_{average}(k))}$;
 - 8: Calculate the velocity of particle x_i according to Eq. (3);
 - 9: Update the position of particle x_i according to Eq. (2);
 - 10: Calculate the fitness of x_i ;
 - 11: Update $pbest_i$ and $gbest$;
 - 12: **end for**
 - 13: $k++$;
 - 14: **end for**
-

results. To further test the performance of the algorithm, we increase the dimension to 40. Algorithms use a uniform asymmetrical initialization in the search space. This initialization method allows the particles to be more randomly distributed in the search space, which is favorable for the population to converge to the globally optimal solution quickly.

4.3. Comparisons of Inertia Weight Diversity

The inertia weight distributions of 20 particles are shown in Fig.3. The horizontal axis represents the iterations, and the vertical axis represents the values of the particle's inertia weight.

Fig.3-a uses the average of the individual's optimal fitness of the particles. When $fitness(pbest)_{average}$ is used, an inertia weight value corresponding to all particles, which cannot balance the abilities of the exploration and exploitation of the algorithm well, will be generated. Fig.3-b shows the result of adjusting the w using the individual optimal fitness of the particles, with inertia weights in the range of $[0.5, 1.5]$, which increases the diversity of inertia weights. This method is conducive to balancing the abilities of exploration and exploitation of the algorithm.

Table 2. Benchmark functions.

	Name	Function	Optimal solution	Search space
f1	Sphere	$f_1(x) = \sum_{i=1}^n x_i^2$	$[-100, 100]^D$	0
f2	Schwefel P2.22	$f_2(x) = \sum_{i=1}^n x_i + \prod_{i=1}^n x_i$	$[-10, 10]^D$	0
f3	Rosenbrock	$f_3(x) = \sum_{i=1}^n [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	$[-30, 30]^D$	0
f4	Noisy Quadric	$f_4(x) = \sum_{i=1}^n ix_i^4 + rand$	$[-1.28, 1.28]^D$	0
f5	Rastrigin	$f_5(x) = \sum_{i=1}^n [x_i^2 - 10\cos(2\pi x_i) + 10]$	$[-5.12, 5.12]^D$	0
f6	Griewank	$f_6(x) = \frac{1}{4000} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos(\frac{x_i}{\sqrt{i}}) + 1$	$[-600, 600]^D$	0
f7	Ackley	$f_7(x) = -20\exp(-0.2\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}) - \exp(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i)) + 20 + e$	$[-32, 32]^D$	0
f8	Rotated hyper ellipsoid	$f_8(x) = \sum_{i=1}^D (\sum_{j=1}^i x_j^2)$	$[-100, 100]^D$	0
f9	Rotated Rastrigin	$f_9(x) = \sum_{i=1}^D [z_i^2 - 10\cos(2\pi z_i) + 10], Z = X \times M$	$[-5, 5]^D$	0
f10	Rotated Griewank	$f_{10}(x) = \frac{1}{4000} \sum_{i=1}^n z_i^2 - \prod_{i=1}^n \cos(\frac{z_i}{\sqrt{i}}) + 1, Z = X \times M$	$[-600, 600]^D$	0
f11	Shifted Rotated Rastrigin	$f_{11}(x) = \sum_{i=1}^n [z_i^2 - 10\cos(2\pi z_i) + 10] - 330, Z = (X - 0) \times M$	$[-600, 600]^D$	-330
f12	Rotated Ackley	$f_{12}(x) = -20\exp(-0.2\sqrt{\frac{1}{n} \sum_{i=1}^n z_i^2}) - \exp(\frac{1}{n} \sum_{i=1}^n \cos(2\pi z_i)) + 20 + e, Z = X \times M$	$[-32, 32]^D$	0
f13	Hybrid Function4	-	$[-100, 100]^D$	2000
f14	Hybrid Function5	-	$[-100, 100]^D$	2100
f15	Hybrid Function6	-	$[-100, 100]^D$	2200
f16	Composition Function1	-	$[-100, 100]^D$	2300
f17	Composition Function2	-	$[-100, 100]^D$	2400
f18	Composition Function3	-	$[-100, 100]^D$	2500

4.4. Comparisons of Solution Accuracy

Table 4 and Table 5 list the worst, best, median, and SD (standard deviation) of the fitness values for all 6 algorithms based on 30-dimensional and 40-dimensional benchmark functions.

In Table 4 and Table 5, min indicates the best solution of the algorithms and is good for evaluating the actual optimal performance of the algorithm; max indicates the worst solution of the algorithms and is good for evaluating the actual worst performance of the algorithm; median represents the median solution of the

Table 3. Parameters of PSO algorithms.

Algorithm	Parameters
CPSO	$c_1 = c_2 = 0.7, w = 1$
RPSO	$c_1 = c_2 = 1.494, w(k) = 0.5 + \frac{rand}{2}$
LDPSO	$c_1 = c_2 = 2, w(k) = 0.4 + (0.9 - 0.4) \frac{(T_{max} - k)}{T_{max}}$
NLDPSO	$c_1 = c_2 = 1.494, w(k) = 0.4 + (0.9 - 0.4) \frac{(T_{max} - k)^2}{(T_{max})^2}$
APSO	$c_1 = c_2 = 1.494, w(k) = 1.1 - \frac{fit(g_{best}(k))}{fit((p_{best})_{average}(k))}$
AIWPSO	$c_1 = c_2 = 1.494, w_i(k) = 0.5rand + \frac{fit(g_{best})(k)}{fit(p_{best})_i(k)}$

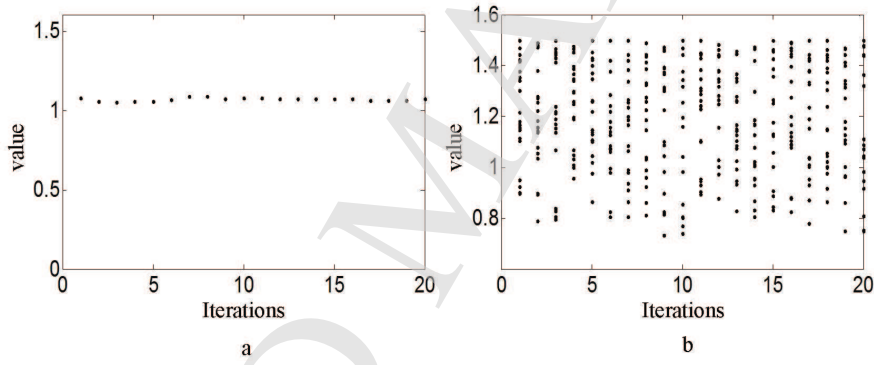


Fig. 3. Distribution of inertia weights.

algorithm and is good for accurately evaluating the actual quality of the solution; SD indicates the standard deviation of the algorithm and is good for evaluating the actual stability of the algorithm. Therefore, we mainly use median to evaluate the results of the 6 algorithms, and the best results of the median in the 6 algorithms are shown in bold. From Table 4, it can be seen that for the 16 benchmark functions, the median of the improved AIWPSO is lower than that of other algorithms (except f_{11} , f_{15}), illustrating that AIWPSO has a better search ability than other algorithms. From Table 5, we can clearly see that when the dimension of the test function increases to 40, for $f_1, f_2, f_3, f_4, f_5, f_6, f_7, f_8, f_9, f_{10}, f_{12}, f_{13}, f_{14}, f_{16}, f_{17}$ and f_{18} functions, the median result of the improved AIWPSO algorithm is lower than that of others, showing that the quality of the improved algorithm is superior to others.

Longitudinal analysis of the simulation results shows that as the dimension of the benchmark function increases, the accuracy decreases. For different test functions, the degree of the decline in accuracy is different.

To compare the quality of different algorithms more intuitively, we compare the results of different algorithms based on a boxplot⁴⁵. The information in the boxplot includes the lower edge (the minimum value of the data), the lower quartile (first quartile), the median, the upper quartile (third quartile), the upper edge (the maximum value of the data) and outliers. In addition, we can analyze the stability of different algorithms through the rectangular box of the optimal fitness. The larger the rectangular box is, the more dispersed the data, indicating that the stability of the algorithm is worse; the smaller the box is, the better the stability. Therefore, the boxplot can clearly reflect the symmetry and distribution of the data.

Fig.4 shows boxplots of different algorithms based on 30-dimensional benchmark test functions. The vertical axis ($\log(\min \text{ fitness})$) represents the best fitness, based on the logarithmic coordinate 10; the horizontal axis represents six algorithms, corresponding to 1 (CPSO), 2 (RPSO), 3 (LDPSO), 4 (NLDPSO), 5 (APSO) and 6 (AIWPSO). The red horizontal line in the boxplot represents the median of the data, and the red plus sign represents the outlier. When the dimension of the test function is 30, for the benchmark test functions $f_1, f_5, f_6, f_8, f_9, f_{10}$ and f_{12} , the median value of the improved AIWPSO algorithm is much lower than that of other algorithms. For the benchmark test functions $f_2, f_3, f_4, f_7, f_{13}, f_{14}, f_{16}, f_{17}$ and f_{18} , the median value of the improved algorithm is also smaller than that of other algorithms, indicating that the quality of the solution of the improved AIWPSO is better. In terms of the stability of the algorithms, when the dimension of test functions is 30, for $f_1, f_3, f_{10}, f_{11}, f_{12}$ and f_{16} , the stability of the AIWPSO algorithm is one of the best. For f_7 , AIWPSO is worse than that of other algorithms, but its accuracy and quality are significantly higher than other algorithms. For the rest of the test functions, the stability of the AIWPSO algorithm is also good.

To further verify the superiority of the AIWPSO, a non-parametric Wilcoxon test is used to statistically test the results of the AIWPSO and the other five algorithms⁴⁶. In this study, the Wilcoxon test has a significance level of 0.05. When the value of the Wilcoxon test is less than 0.05, it shows that there is a significant difference between the AIWPSO and the other algorithms; otherwise, there is no

significant difference. Based on the 30-dimensional and 40-dimensional benchmark functions ($f_1 - f_{18}$), the values of the Wilcoxon tests for the AIWPSO and the other PSOs are shown in Tables 6 and 7.

It can be seen that there is a significant difference between the experimental result of the AIWPSO and the experimental results of the other algorithms on benchmark functions (except f_{13} , f_{14} , f_{15}) based on 30 and 40 dimensions, demonstrating that AIWPSO is significantly better than other algorithms on f_1 , f_2 , f_3 , f_4 , f_5 , f_6 , f_7 , f_8 , f_9 , f_{10} , f_{12} , f_{16} , f_{17} and f_{18} , but is significantly worse than other algorithms on f_{11} .

We also compare the results with the PSO algorithms in other references. Alireza uses three benchmark functions including Rosenbrock, Rastrigin and Griewank, and our PSO algorithm has higher accuracy and search speed than its results⁴⁷. Fernandes et al.⁴⁸ uses four benchmark functions. For the Sphere function, our proposed method is less precise. For Rosenbrock, Rastrigin and Griewank functions, the accuracy of our method is higher⁴⁸. The experiment uses the same parameters as the AIWPSO, and the results are shown in Table 8.

4.5. Comparisons of Convergence Speed

Fig.5 shows the mean optimal fitness curves for different PSO algorithms in a 30-dimensional search space based on 18 benchmark functions^{49,50,51}. The thick and solid lines are the optimal fitness curves of the improved algorithm.

In Fig.5, the horizontal axis represents the number of iterations, and the vertical axis represents the mean optimal fitness of the algorithm. Each curve represents the change of the optimal fitness in iterations. The end point of each curve represents the global optimum. It can be seen from Fig.5 that the convergence speed of the improved algorithm is fast, for example, for functions f_1 , f_2 , f_8 , f_{10} , f_{12} , f_{16} , f_{17} and f_{18} . For other functions (except f_{11} , f_{15}), the convergence speed is slower, but the ability to jump out of the local optimum is enhanced.

To compare the computational cost of different algorithms, Table 9 gives the mean iterations and time for each algorithm to achieve an acceptable solution based on 30 dimensions. For null values in Table 9, the algorithm for the corresponding test functions does not achieve acceptable solution. The least loss of time is in bold fonts. It can be concluded that the improved AIWPSO algorithm uses the fewest number

of iterations in the f_1 , f_2 , f_4 , f_5 , f_8 , and f_9 functions and has the fastest convergence speed. For the rest of the test functions, the computational cost is slightly higher. The reason may be that on the one hand, the calculation will affect the time. On the other hand, the time may be related to the characteristics of functions themselves. For example, the computational cost of the CPSO on function f_3 (Rosenbrock) is better than AIWPSO which may be connected with characteristics of f_3 . f_3 is an indivisible multimodal function and has a very narrow valley from local optimum to global optimum, which makes the algorithm easy to fall into local optimum. In addition, the inertia weight in the CPSO is a fixed value ($w=1$), so that the particle can jump out of the current local optimum at a large speed and converge to a better solution more quickly.

5. Concluding Remarks

In this paper, a new adaptive inertia weight PSO algorithm (AIWPSO) is proposed. The algorithm has an excellent performance and uses two strategies. The first strategy is an inertia weight adjustment method based on the individual optimal fitness of particles, which can increase the diversity of the inertia weights. For particles, different weights are used to affect the speed of the next generation; when w is a small value, the particle's velocity changes less, and the particle tends to exploit the landscape at the current position. When w is larger, the particle's velocity changes significantly. These particles tend to explore in a larger range. This strategy helps to balance the abilities of exploration and exploitation. The second strategy is to use the mean value of the Euclidean distance from the location of the successful particle to the global optimum location as the mutation threshold to determine which particles need to be mutated. This strategy compensates for the limitations of random mutation, thus effectively increasing the diversity of the population. This paper uses benchmark functions to test the performance of the AIWPSO algorithm. The results show that compared with other PSO algorithms, AIWPSO achieves satisfactory results. This shows that the AIWPSO algorithm helps to balance the abilities of the global exploration and local exploitation of the population, and increases the diversity of the population, thus significantly improving the optimization of PSO.

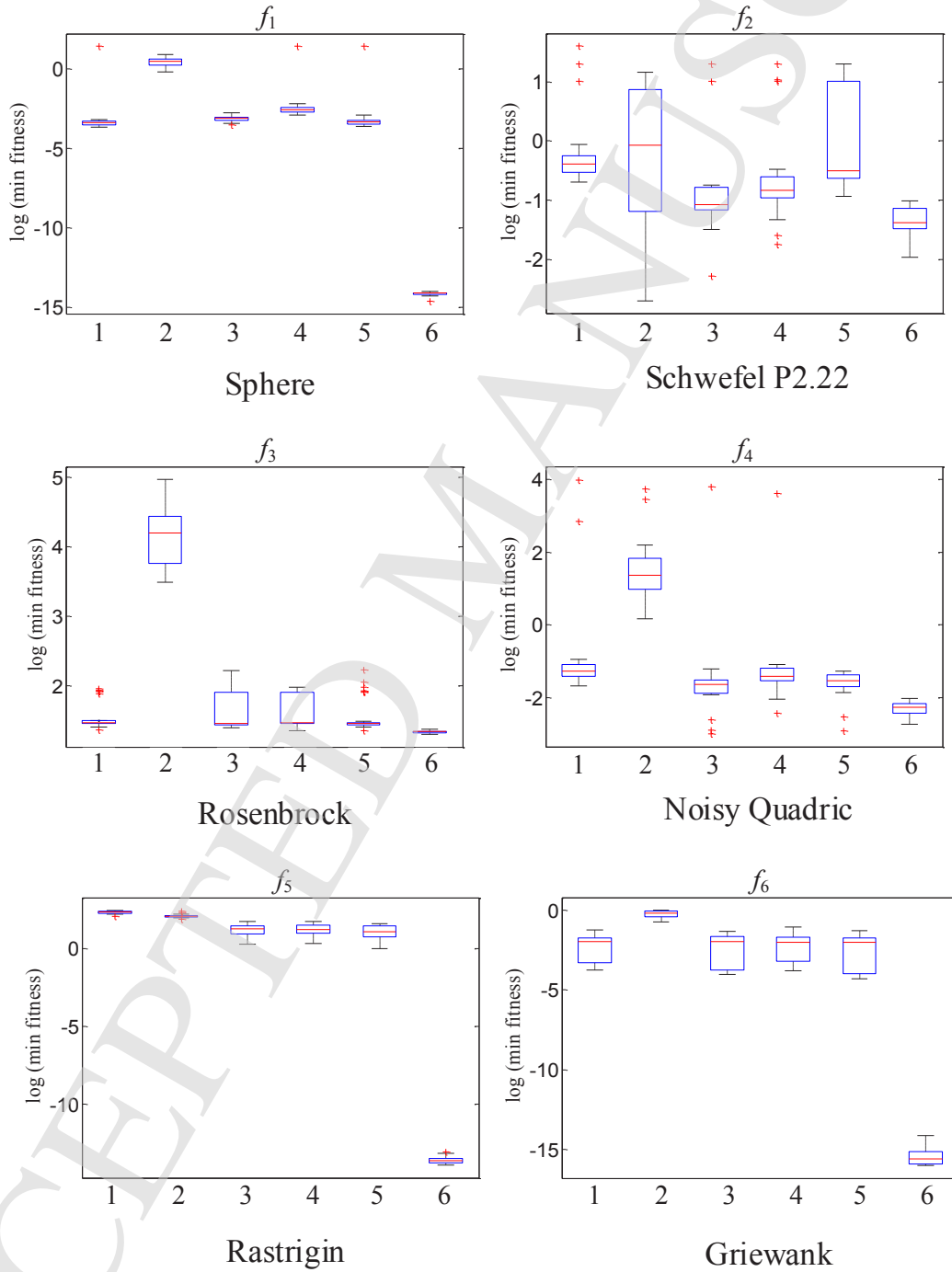
In the 18 benchmark function tests, the AIWPSO algorithm far outperforms

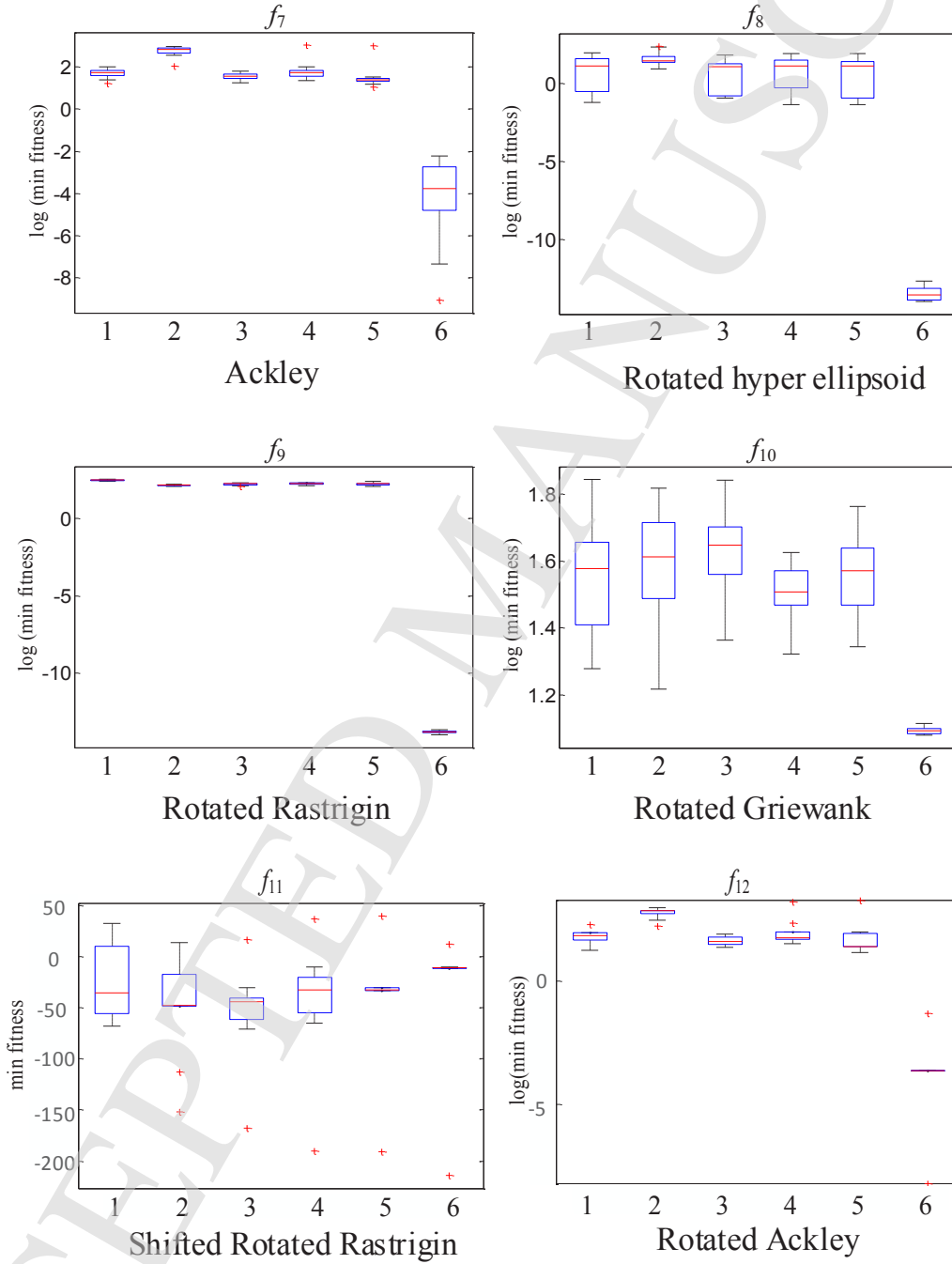
other comparison algorithms in 16 functions and is only equivalent to the results of other algorithms on the “Shifted Rotated Rastrigin” and “Hybrid Function6”. In the PSO, in addition to the inertia weight, the acceleration factor and the topology structure of the particles can also be used to improve the optimization performance of the algorithm. Therefore, for such functions, we will further study the improvement of the algorithm in future work, to enhance the applicability of the algorithm. In addition to function optimization, the AIWPSO algorithm proposed can also be used to optimize related parameters in the machine recognition model, such as support vector machine, artificial neural network, extreme learning machine and so on, so as to improve the accuracy of correlation classification. At the same time, the algorithm can also be applied to fuzzy system control, logistics dispatch system, financial time series forecasting, evaluation of real estate investment alternatives, etc., to further optimize the performance of the system.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (61602017), the National Basic Research Programme of China (2014CB744600), ‘Rixin Scientist’ Foundation of Beijing University of Technology (2017-RX(1)-03), the Beijing Natural Science Foundation (4164080), the Beijing Outstanding Talent Training Foundation (2014000020124G039), the National Natural Science Foundation of China (61420106005), the International Science & Technology Cooperation Program of China (2013DFA32180), the Special fund of Beijing Municipal Science and Technology Commission (Z171100000117004 and Z151100003915117), Beijing Municipal Administration of Hospitals Clinical Medicine Development of Special Funding Support (ZYLX201607), and Beijing Municipal Administration of Hospitals Ascent Plan (DFL20151801).

Algorithms: 1 (CPSO), 2 (RPSO), 3 (LDPSO), 4 (NLDPSO), 5 (APSO) and 6 (AIWPSO)





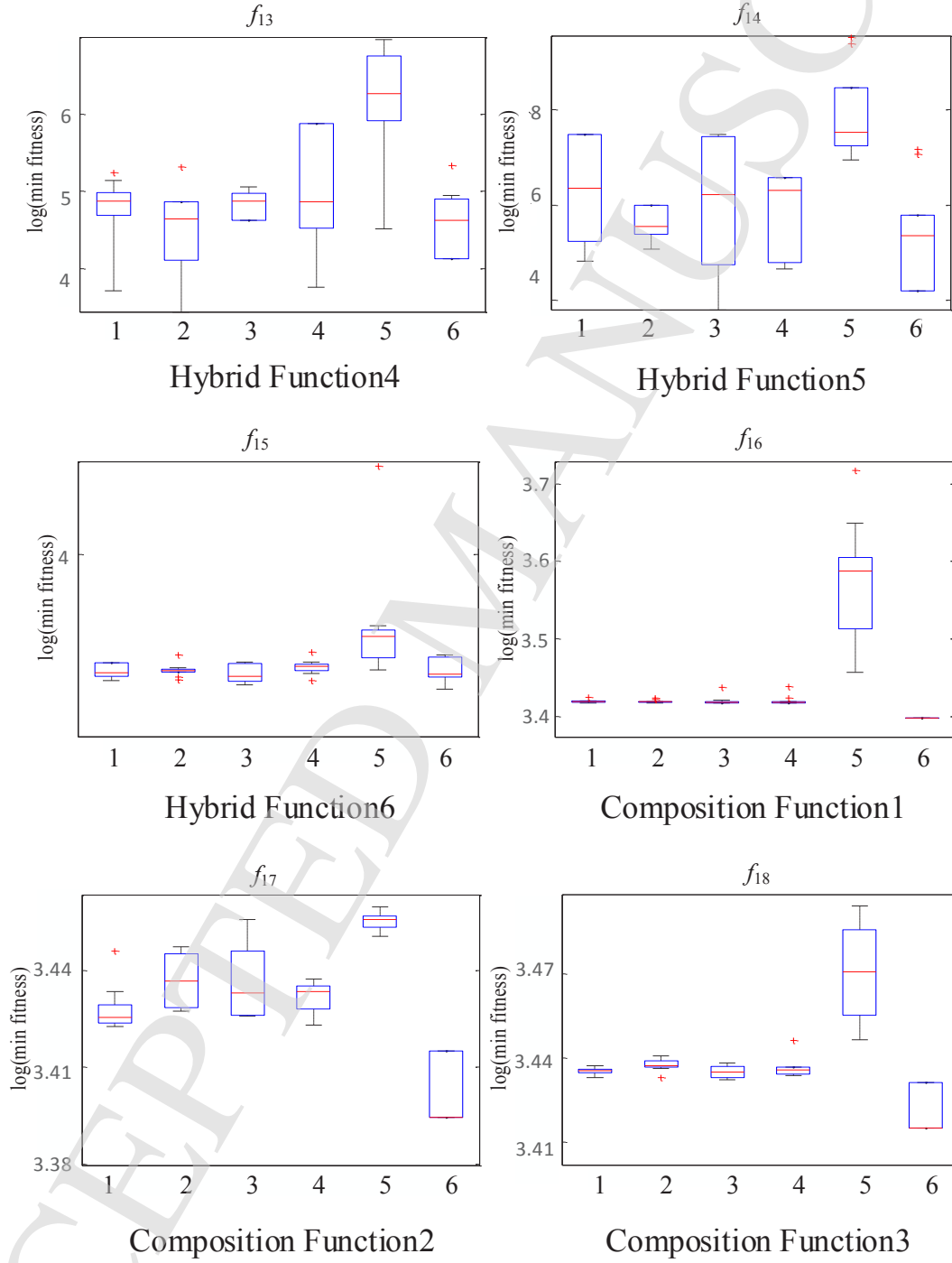
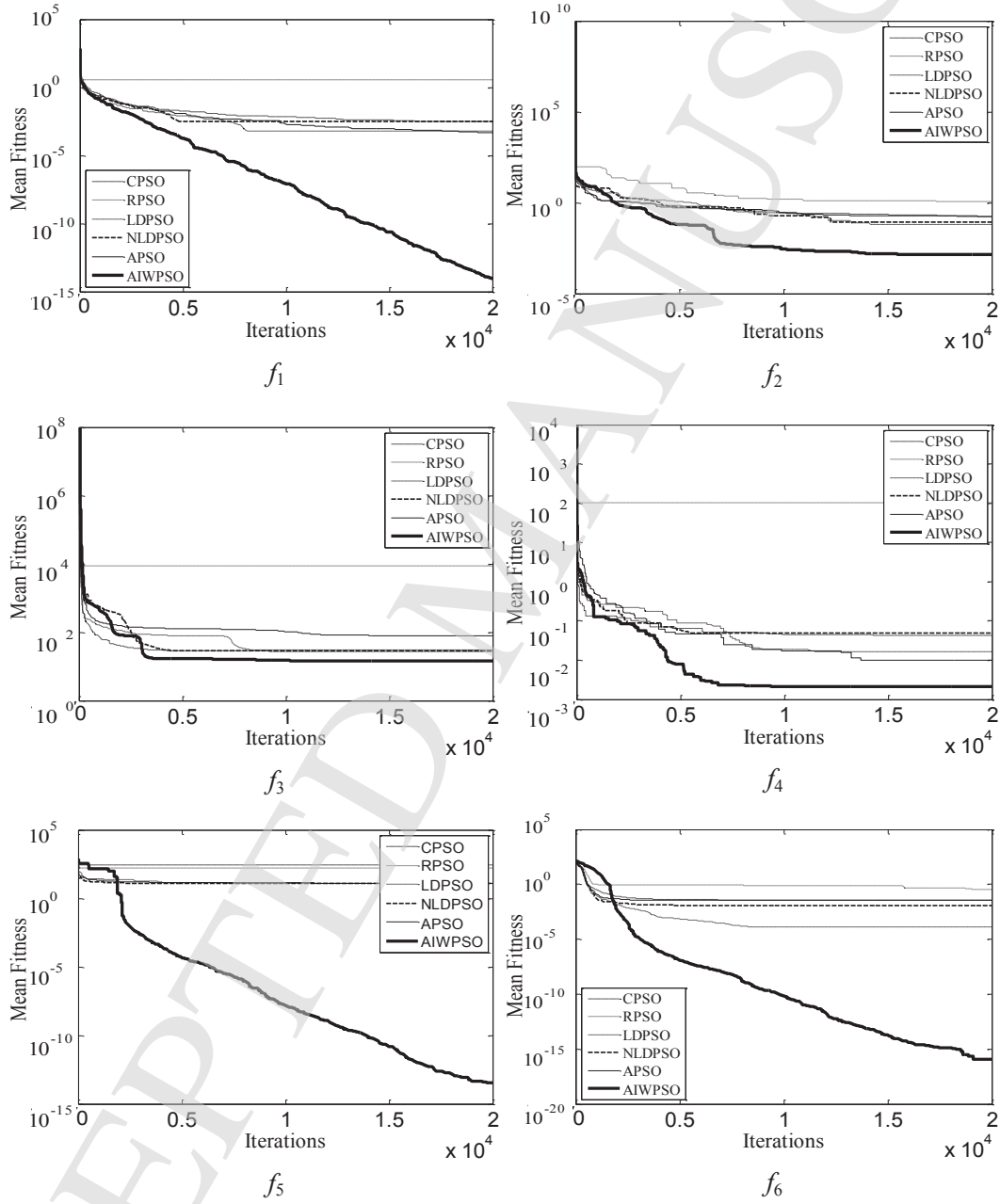
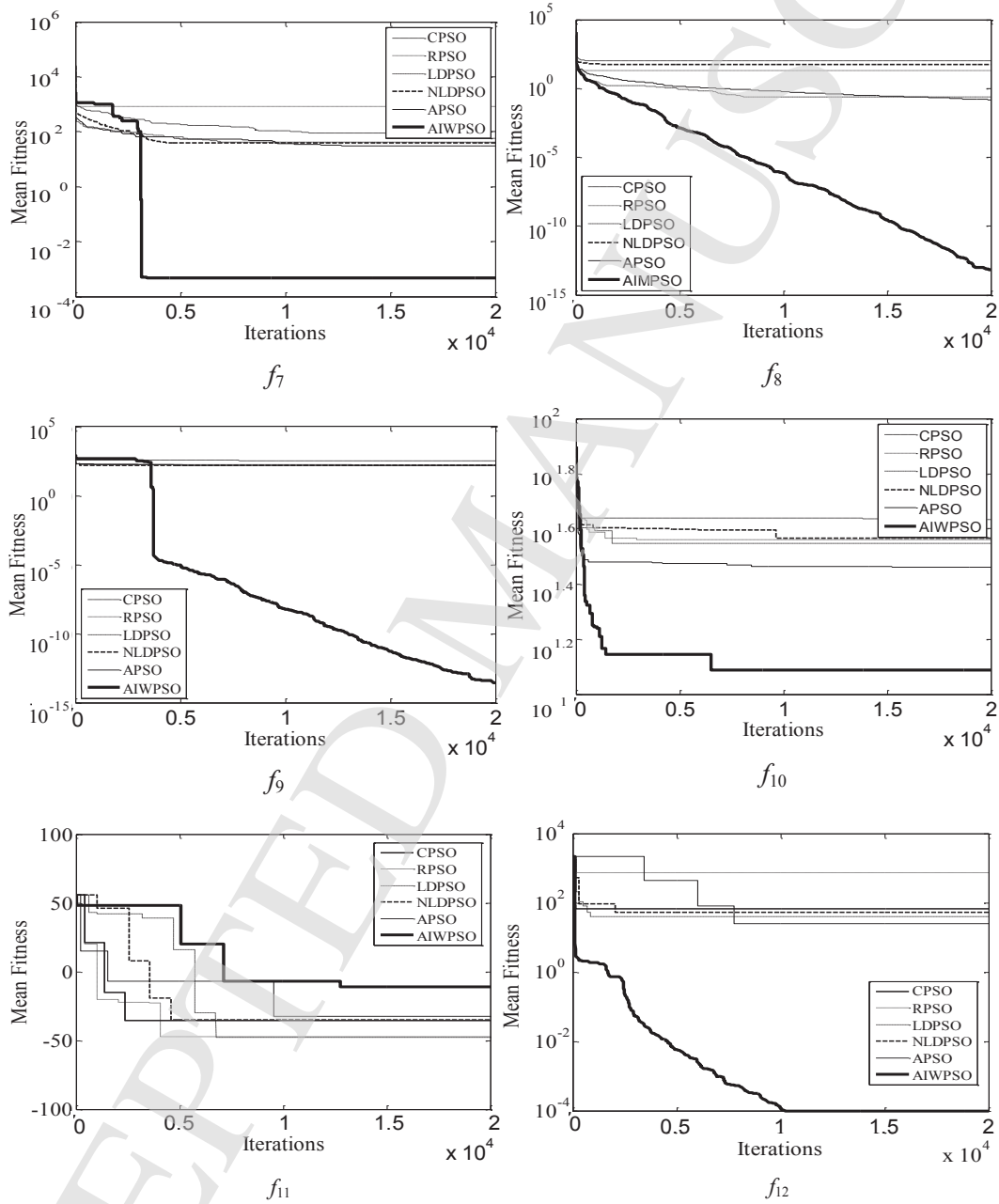


Fig. 4. Box plots of the results of the different algorithms based on 30 dimensional benchmark functions ($f_1 - f_{18}$).





30

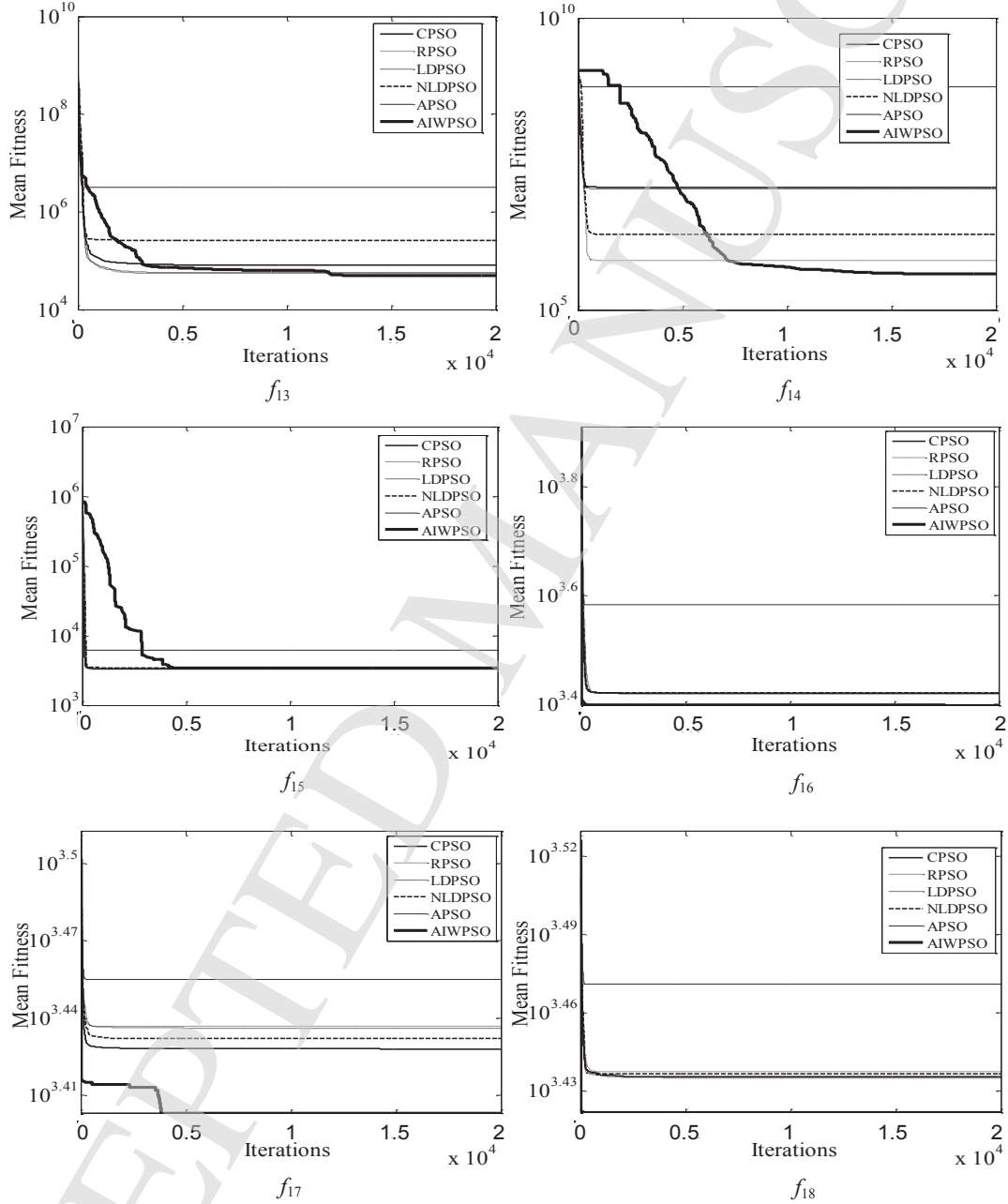


Fig. 5. Curves of the mean optimal fitness of CPSO, RPSO, LDPSO, NLDPSO, APSO and AIWPSO on the 18 benchmark functions.

References

1. J. Kennedy, R. Eberhart, "Particle swarm optimization," *IEEE International Conference on Neuron Networks, Proceedings*, vol. 4, pp. 1942-1948, 1995.
2. Y.D. Valle, G.K. Venayagamoorthy, S. Mohagheghi, et al., "Particle swarm optimization: basic concepts, variants and applications in power systems," *Evolutionary Computation, IEEE Transactions on*, vol. 12, Issue 2, pp. 171-195, 2008.
3. C.J. Lin, M.H. Hsieh, "Classification of mental task from EEG data using neural networks based on particle swarm optimization," *Neurocomputing*, vol. 72, Issue 4, pp. 1121-1130, 2009.
4. R.J. Wai, J.D. Lee, K.L. Chuang, "Real-time PID control strategy for maglev transportation system via particle swarm optimization," *Industrial Electronics, IEEE Transactions on*, vol. 58, Issue 2, pp. 629-646, 2011.
5. W.C. Wang, et al., "Improved annual rainfall-runoff forecasting using PSO-SVM model based on EEMD," *Journal of Hydroinformatics*, vol. 15, Issue 4, pp. 1377-1390, 2013.
6. K.W. Chau, "A split-step particle swarm optimization algorithm in river stage forecasting," *Journal of Hydrology*, vol. 346, Issue 3-4, pp. 131-135, 2007.
7. S. Zhang, K.W. Chau, "Dimension Reduction Using Semi-Supervised Locally Linear Embedding for Plant Leaf Classification," *International Conference on Emerging Intelligent Computing Technology and Applications. Springer-Verlag*, pp. 948-955, 2009.
8. V. Gholami, K.W. Chau, F. Fadaee, et al., "Modeling of groundwater level fluctuations using dendrochronology in alluvial aquifers," *Journal of Hydrology*, vol. 529, pp. 1060-1069, 2015.
9. R. Taormina, K.W. Chau, B. Sivakumar, "Neural network river forecasting through baseflow separation and binary-coded swarm optimization," *Journal of Hydrology*, vol. 529, pp. 1788-1797, 2015.
10. F. Fotovatikhah, et al., "Survey of Computational Intelligence as Basis to Big Flood Management: Challenges, research directions and Future Work," *Engineering Applications of Computational Fluid Mechanics*, vol. 12, Issue 1, 2018.
11. S. Pang, S. Li, J. Xiao, "Application of the algorithm based on the PSO and improved SVDD for the personal credit rating," *Journal of Financial Engineering*, vol. 1, Issue 04, pp. 1450037, 2014.
12. M. Khashei, Z. Hajirahimi, "Performance evaluation of series and parallel strategies for financial time series forecasting," *Financial Innovation*, vol. 3, Issue 1, pp. 24, 2017.
13. G. Kou, D. Ergu, C. Lin, et al., "Pairwise comparison matrix in multiple criteria decision making," *Technological and Economic Development of Economy*, vol. 22, Issue 5, pp. 738-765, 2016.
14. G. Kou, D. Ergu, J. Shang, "Enhancing data consistency in decision matrix: Adapting Hadamard model to mitigate judgment contradiction," *European Journal of Opera-*

- tional Research*, vol. 236, Issue 1, pp. 261-271, 2014.
15. W. Wu, G. Kou, "A group consensus model for evaluating real estate investment alternatives," *Financial Innovation*, vol. 2, Issue 1, pp. 8, 2016.
 16. A. Nickabadi, M.M. Ebadzadeh, R. Safabakhsh, "A novel particle swarm optimization algorithm with adaptive inertia weight," *Applied Soft Computing*, vol. 11, Issue 4, pp. 3658-3670, 2011.
 17. C.M. Fernandes, J.J. Merelo, A.C. Rosa, "Controlling the parameters of the particle swarm optimization with a self-organized criticality model," vol. 7492, pp. 153-163, 2012.
 18. Q. Wu, "Power load forecasts based on hybrid PSO with Gaussian and adaptive mutation and Wv-SVM," *Pergamon Press, Inc*, 2010.
 19. S.H. Ling, H.H. Iu, K.Y. Chan, et al., "Hybrid particle swarm optimization with wavelet mutation and its industrial applications," *IEEE Transactions on Systems Man & Cybernetics Part B Cybernetics A Publication of the IEEE Systems Man & Cybernetics Society*, vol. 38, Issue 3, pp. 743-763, 2008.
 20. P. Melin, F. Olivas, O. Castillo, et al., "Optimal design of fuzzy classification systems using PSO with dynamic parameter adaptation through fuzzy logic," *Expert Systems with Applications. An International journal*, vol. 40, Issue 8, pp. 3196-3206, 2013.
 21. Y. Maldonado, O. Castillo, P. Melin, "Particle swarm optimization of interval type-2 fuzzy systems for FPGA applications," *Applied Soft Computing*, vol. 13, Issue 1, pp. 496-508, 2013.
 22. J. Sun, W. Fang, X. Wu, et al., "Quantum-behaved particle swarm optimization: Analysis of individual particle behavior and parameter selection," *Evolutionary computation*, vol. 20, Issue 3, pp. 349-393, 2014.
 23. X. Xu, et al., "Dynamic multi-swarm particle swarm optimizer with cooperative learning strategy," *Applied Soft Computing*, vol. 29, pp. 169-183, 2015.
 24. Y. Shi, R. Eberhart, "Modified particle swarm optimizer," *Proc. of IEEE ICEC conference, Anchorage*, pp. 69-73, 1998.
 25. C. Pornsing, M.S. Sodhi, B.F. Lamond, "Novel self-adaptive particle swarm optimization methods," *Soft Computing*, vol. 20, Issue 9, pp. 3579-3593, 2016.
 26. V.D.B. Frans, "An analysis of particle swarm optimizers," *Phd Thesis*, 2006.
 27. R.C. Eberhart, Y.H. Shi, "Comparing inertia weights and constriction factors in particle swarm optimization," *Proc. of the 2000 Congress on Evolutionary Computation*, vol. 1, pp. 84-88, 2000.
 28. A. Chatterjee, P. Siarry, "Nonlinear inertia weight variation for dynamic adaption in particle swarm optimization," *Computer and Operations Research*, vol. 33, Issue 3, pp. 859-871, 2006.
 29. X. Yang, J. Yuan, H. Mao, "A modified particle swarm optimizer with dynamic adaptation," *Applied Mathematics and Computation*, vol. 189, Issue 2, pp. 1205-1213, 2007.

30. M.S. Arumugam, M.V.C. Rao, "On the improved performances of the particle swarm optimization algorithms with adaptive parameters, cross-over operators and root mean square (RMS) variants for computing optimal control of a class of hybrid systems," *Applied Soft Computing*, vol. 8, Issue 1, pp. 324-336, 2008.
31. L. Zhang, Y. Tang, C. Hua, et al., "A new particle swarm optimization algorithm with adaptive inertia weight based on Bayesian techniques," *Applied Soft Computing*, vol. 28, Issue C, pp. 138-149, 2015.
32. W. Hu, G.G. Yen, "Adaptive multiobjective particle swarm optimization based on parallel cell coordinate system," *Evolutionary Computation, IEEE Transactions on*, vol. 19, Issue 1, pp. 1-18, 2015.
33. A. Ratnaweera, S.K. Halgamuge, H.C. Watson, "Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients," *IEEE Transactions on evolutionary computation*, vol. 8, Issue 3, pp. 240-255, 2004.
34. Q. Bai, "Analysis of particle swarm optimization algorithm," *Computer and Information Science*, vol. 3, Issue 1, pp. 180-184, 2010.
35. Y. Feng, G. Teng, A. Wang, Y.M. Yao, "Chaotic inertia weight in particle swarm optimization," *International Conference on Innovative Computing, Information and Control. IEEE Computer Society*, pp. 475, 2007.
36. S.K.S. Fan, Y.Y. Chiu, "A decreasing inertia weight particle swarm optimizer, engineering optimization," *Engineering Optimization*, vol. 39, Issue 2, pp. 203-228, 2007.
37. B. Jiao, Z. Lian, X. Gu, "A dynamic inertia weight particle swarm optimization algorithm," *Chaos, Solitons, Fractals*, vol. 37, Issue 3, pp. 698-705, 2008.
38. B.K. Panigrahi, V.R. Pandi, S. Das, "Adaptive particle swarm optimization approach for static and dynamic economic load dispatch," *Energy Conversion and Management*, vol. 49, Issue 6, pp. 1407-1415, 2008.
39. K. Lei, Y. Qiu, Y. He, "A new adaptive well-chosen inertia weight strategy to automatically harmonize global and local search ability in particle swarm optimization," *International Symposium on Systems and Control in Aerospace and Astronautics IEEE*, vol. 4, pp. 980, 2006.
40. Z. Qin, F. Yu, Z. Shi, Y. Wang, "Adaptive inertia weight particle swarm optimization," *Lecture Notes in Computer Science*, vol. 4029, pp. 450-459, 2006.
41. N. Higashi, H. Lba, "Particle swarm optimization with Gaussian mutation," in *Swarm Intelligence Symposium 2003. SIS'03. Proceedings of the IEEE*, pp. 72-79, 2003.
42. R.C. Eberhart, Y.H. Shi, "Tracking and optimizing dynamic systems with particle swarms," *Evolutionary Computation, 2001. Proceedings of the 2001 Congress on IEEE*, vol. 1, pp. 94-100, 2001.
43. K. Tang, X. Li, P.N. Suganthan, et al., "Benchmark Functions for the CEC'2010 Special Session and Competition on Large-Scale Global Optimization," *Nature Inspired Computation & Applications Laboratory*, 2013.

44. J.J. Liang, B. Y. Qu, and P. N. Suganthan, "Problem definitions and evaluation criteria for the CEC 2014 special session and competition on single objective real-parameter numerical optimization," *Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou China and Technical Report, Nanyang Technological University, Singapore*, 2013.
45. X. Yu, J. Cao, H. Shan, et al., "An adaptive hybrid algorithm based on particle swarm optimization and differential evolution for global optimization," *The Scientific World Journal*, vol. 6, pp. 215472, 2014.
46. J.J. Liang, A.K. Qin, P.N. Suganthan, et al., "Comprehensive learning particle swarm optimizer for global optimization of multimodal functions," *IEEE Transactions on Evolutionary Computation*, vol. 10, Issue 3, pp. 281-295, 2006.
47. A. Alireza, "PSO with adaptive mutation and inertia weight and its application in parameter estimation of dynamic systems," *Acta Automatica Sinica*, vol. 37, Issue 5, pp. 541-549, 2011.
48. C.M. Fernandes, J.J. Merelo, A.C. Rosa, "A time-varying inertia weight strategy for particles swarms based on self-organized criticality," *Computational Intelligence. Springer International Publishing*, pp. 49-62, 2015.
49. C. Pornsing, M.S. Sodhi, B.F. Lamond, "Novel self-adaptive particle swarm optimization methods," *Soft Computing*, vol. 20, Issue 9, pp. 3579-3593, 2016.
50. K. Tang, Z. Li, L. Luo, et al., "Multi-strategy adaptive particle swarm optimization for numerical optimization," *Engineering Applications of Artificial Intelligence*, vol. 37, pp. 9-19, 2015.
51. B. Niu, H. Huang, L. Tan, et al., "Symbiosis-Based Alternative Learning Multi-Swarm Particle Swarm Optimization," *IEEE/ACM Trans Comput Biol Bioinform*, vol. 14, Issue 1, pp. 4-14, 2017.

Table 4. Comparisons and analysis of the search results of the different algorithms based on benchmark functions with 30 dimensions ($f_1 - f_{18}$).

		CPSO	RPSO	LDPSO	NLDPSO	APSO	AIWPSO
f1	Min	2.12E-04	6.73E-01	2.99E-04	1.20E-03	2.55E-04	2.59E-15
	Max	2.62E+01	8.18E+00	1.80E-03	2.62E+01	2.62E+01	9.54E-15
	Median	4.24E-04	3.13E+00	8.27E-04	2.60E-03	4.73E-04	7.14E-15
	SD	4.71E+00	1.91E+00	2.95E-04	6.54E+00	4.71E+00	1.35E-15
f2	Min	2.05E-01	2.00E-02	5.20E-02	1.76E-02	1.15E-01	1.09E-02
	Max	4.02E+01	1.46E+01	2.03E+01	2.04E+01	2.01E+01	9.87E-02
	Median	4.11E-01	8.82E-01	8.46E-02	1.48E-01	3.15E-01	4.20E-02
	SD	8.19E+00	4.89E+00	5.03E+00	6.36E+00	5.50E+00	2.63E-02
f3	Min	2.26E+01	3.10E+03	2.47E+01	2.25E+01	2.22E+01	1.99E+01
	Max	9.10E+01	9.32E+04	1.64E+02	9.44E+01	1.67E+02	2.34E+01
	Median	2.94E+01	1.56E+04	2.87E+01	2.93E+01	2.82E+01	2.18E+01
	SD	2.33E+01	2.18E+04	3.94E+01	2.67E+01	3.33E+01	9.60E-01
f4	Min	2.09E-02	1.49E+00	1.00E-03	3.80E-03	1.20E-03	1.90E-03
	Max	9.62E+03	5.51E+03	6.18E+03	4.12E+03	5.46E-02	9.40E-03
	Median	5.35E-02	2.32E+01	2.36E-02	3.94E-02	2.88E-02	5.45E-03
	SD	1.73E+03	1.09E+03	1.11E+03	7.40E+02	1.36E-02	2.12E-03
f5	Min	1.24E+02	8.12E+01	1.99E+00	2.17E+00	1.03E+00	1.24E-14
	Max	3.03E+02	2.61E+02	5.89E+01	5.80E+01	4.33E+01	8.53E-14
	Median	2.34E+02	1.26E+02	1.90E+01	1.74E+01	1.25E+01	2.40E-14
	SD	4.92E+01	3.34E+01	1.62E+01	1.50E+01	1.42E+01	1.56E-14
f6	Min	1.67E-04	1.74E-01	8.85E-05	1.52E-04	5.12E-05	1.01E-16
	Max	5.93E-02	9.63E-01	4.69E-02	8.63E-02	5.18E-02	7.40E-15
	Median	1.02E-02	6.22E-01	1.00E-02	9.40E-03	9.90E-03	2.58E-16
	SD	1.69E-02	2.45E-01	1.35E-02	2.12E-02	1.43E-02	1.41E-15
f7	Min	1.70E+01	1.13E+02	1.80E+01	2.36E+01	1.12E+01	8.70E-10
	Max	1.05E+02	9.41E+02	6.44E+01	1.09E+03	1.03E+03	6.00E-03
	Median	5.60E+01	6.73E+02	3.55E+01	5.40E+01	2.51E+01	1.72E-04
	SD	2.17E+01	1.88E+02	1.20E+01	1.87E+02	1.81E+02	1.81E-03
f8	Min	6.10E-02	8.95E+00	1.12E-01	4.52E-02	4.52E-02	1.01E-14
	Max	9.18E+01	2.43E+02	6.56E+01	8.32E+01	8.30E+01	2.18E-13
	Median	1.34E+01	2.79E+01	1.19E+01	1.36E+01	1.32E+01	2.62E-14
	SD	2.27E+01	5.22E+01	1.47E+01	2.38E+01	2.25E+01	4.72E-14
f9	Min	2.40E+02	1.10E+02	1.12E+02	1.25E+02	1.14E+02	1.10E-14
	Max	3.34E+02	1.63E+02	2.00E+02	2.05E+02	2.43E+02	2.31E-14
	Median	2.97E+02	1.40E+02	1.69E+02	1.73E+02	1.71E+02	1.67E-14
	SD	2.71E+01	1.24E+01	1.86E+01	2.34E+01	3.03E+01	2.99E-15
f10	Min	1.90E+01	1.65E+01	2.30E+01	2.10E+01	2.20E+01	1.20E+01
	Max	6.97E+01	6.57E+01	6.92E+01	4.21E+01	5.80E+01	1.30E+01
	Median	3.77E+01	4.09E+01	4.43E+01	3.21E+01	3.72E+01	1.23E+01
	SD	1.22E+01	1.33E+01	1.25E+01	5.96E+00	8.64E+00	2.66E-01
f11	Min	-6.79E+01	-1.52E+02	-1.68E+02	-1.90E+02	-1.91E+02	-2.14E+02
	Max	3.23E+01	1.37E+01	1.65E+01	3.67E+01	3.98E+01	1.22E+01
	Median	-3.54E+01	-4.76E+01	-4.73E+01	-3.51E+01	-3.22E+01	-1.09E+01
	SD	2.56E+01	1.55E+01	1.36E+01	1.83E+01	1.14E+01	1.03E+01
f12	Min	1.83E+01	1.75E+02	2.37E+01	3.42E+01	1.46E+01	6.21E-09
	Max	1.98E+02	9.87E+02	8.04E+01	1.75E+03	1.92E+03	5.06E-02
	Median	7.14E+01	7.65E+02	4.23E+01	5.68E+01	2.63E+01	2.43E-04
	SD	2.34E+01	2.07E+02	1.76E+01	1.92E+02	1.93E+02	2.07E-03
f13	Min	5.24E+03	2.75E+03	4.24E+04	5.79E+03	3.29E+04	1.36E+04
	Max	1.75E+05	2.06E+05	1.15E+05	7.59E+05	9.12E+06	2.14E+05
	Median	7.46E+04	4.41E+04	7.58E+04	7.34E+04	1.84E+06	4.27E+04
	SD	4.85E+04	5.86E+04	2.57E+04	3.44E+05	3.05E+06	6.15E+04
f14	Min	6.55E+04	1.18E+05	6.19E+03	4.48E+04	8.88E+06	1.54E+04
	Max	3.03E+07	9.69E+05	3.08E+07	3.80E+06	3.39E+09	1.46E+06
	Median	2.25E+06	3.51E+05	1.64E+06	2.02E+06	3.36E+07	2.22E+05
	SD	1.52E+07	3.51E+05	1.47E+07	1.93E+06	1.23E+09	5.54E+06
f15	Min	3.04E+03	3.07E+03	2.91E+03	3.03E+03	3.36E+03	2.81E+03
	Max	3.59E+03	3.86E+03	3.63E+03	3.99E+03	2.31E+04	3.86E+03
	Median	3.27E+03	3.34E+03	3.17E+03	3.48E+03	4.61E+03	3.23E+03
	SD	2.23E+02	2.11E+02	2.80E+02	2.47E+02	5.98E+03	3.86E+02
f16	Min	2.62E+03	2.62E+03	2.62E+03	2.62E+03	2.87E+03	2.50E+03
	Max	2.66E+03	2.65E+03	2.74E+03	2.75E+03	5.22E+03	2.50E+03
	Median	2.62E+03	2.62E+03	2.62E+03	2.62E+03	3.87E+03	2.50E+03
	SD	1.25E+01	1.16E+01	3.75E+01	4.05E+01	6.89E+02	6.21E-04
f17	Min	2.65E+03	2.67E+03	2.67E+03	2.65E+03	2.82E+03	2.48E+03
	Max	2.79E+03	2.80E+03	2.86E+03	2.74E+03	2.88E+03	2.60E+03
	Median	2.66E+03	2.73E+03	2.71E+03	2.86E+03	2.86E+03	2.48E+03
	SD	4.44E+01	5.01E+01	6.62E+01	2.83E+01	1.72E+01	6.20E+01
f18	Min	2.71E+03	2.71E+03	2.71E+03	2.71E+03	2.80E+03	2.60E+03
	Max	2.74E+03	2.76E+03	2.74E+03	2.80E+03	3.12E+03	2.70E+03
	Median	2.73E+03	2.74E+03	2.72E+03	2.73E+03	2.96E+03	2.60E+03
	SD	7.55E+00	1.29E+01	1.37E+01	2.31E+01	1.14E+02	5.16E+01

Table 5. Comparisons and analysis of the search results of the different algorithms based on benchmark functions with 40 dimensions ($f_1 - f_{18}$).

		CPSO	RPSO	LDPSO	NLDPSO	APSO	AIWPSO
f1	Min	5.40E-03	1.45E+00	3.20E-03	6.10E-03	8.58E-04	8.32E-15
	Max	2.62E-02	2.93E+01	2.62E+01	3.56E-02	3.00E-03	2.62E-14
	Median	1.33E-02	5.38E+00	4.65E-03	1.21E-02	1.90E-03	1.24E-14
	SD	4.52E-03	5.92E+00	6.54E+00	6.01E-03	5.36E-04	3.25E-15
f2	Min	6.66E-01	3.22E-02	5.94E-02	2.27E-01	2.82E-01	1.04E-02
	Max	3.10E+01	1.13E+02	2.13E+01	3.14E+01	5.02E+01	6.35E-01
	Median	5.44E+00	7.11E+00	2.96E-01	1.05E+01	5.48E+00	1.09E-01
	SD	8.08E+00	2.30E+01	5.58E+00	9.55E+00	1.00E+01	1.57E-01
f3	Min	3.62E+01	3.58E+03	3.41E+01	3.92E+01	3.06E+01	3.03E+01
	Max	1.38E+02	8.72E+04	1.68E+02	4.32E+02	1.88E+02	3.59E+01
	Median	4.20E+01	3.33E+04	4.01E+01	7.62E+01	3.95E+01	3.34E+01
	SD	3.47E+01	2.37E+04	4.07E+01	8.80E+01	3.54E+01	1.04E+00
f4	Min	4.67E-02	1.07E+01	1.06E-02	6.51E-02	3.54E-02	1.02E-02
	Max	1.47E+03	7.82E+02	2.38E-01	6.87E-01	1.33E-01	9.80E-02
	Median	1.76E-01	1.37E+02	9.21E-02	1.41E-01	6.87E-02	1.61E-02
	SD	2.63E+02	1.89E+02	4.32E-02	1.07E-01	2.35E-02	2.90E-02
f5	Min	2.15E+02	1.24E+02	8.41E+00	4.85E+00	5.22E+00	1.07E-14
	Max	4.30E+02	2.89E+02	8.82E+01	7.53E+01	9.58E+01	1.25E-12
	Median	3.21E+02	2.07E+02	5.05E+01	3.64E+01	3.91E+01	1.17E-13
	SD	6.01E+01	4.01E+01	2.09E+01	1.86E+01	2.57E+01	2.87E-13
f6	Min	1.00E-03	1.12E-01	9.36E-05	1.02E-02	1.47E-04	1.02E-15
	Max	3.60E-01	1.03E+00	1.78E-02	3.37E-01	7.70E-04	7.60E-15
	Median	7.20E-03	5.44E-01	5.37E-04	3.26E-02	3.03E-04	4.01E-15
	SD	6.47E-02	2.68E-01	5.19E-03	7.09E-02	1.12E-04	2.20E-15
f7	Min	4.45E+01	5.21E+02	4.70E+01	5.78E+01	3.00E+01	1.06E-03
	Max	1.12E+03	1.83E+03	1.16E+03	1.85E+02	6.52E+01	9.29E-03
	Median	1.19E+02	9.72E+02	7.51E+01	1.13E+02	3.94E+01	2.63E-03
	SD	1.86E+02	3.13E+02	1.96E+02	3.35E+01	9.19E+00	2.26E-03
f8	Min	1.01E+01	1.28E+02	1.10E+01	3.24E+00	5.66E+00	1.10E-12
	Max	9.92E+01	8.97E+02	5.55E+02	9.02E+01	8.32E+01	8.89E-12
	Median	3.96E+01	4.37E+02	2.12E+02	3.79E+01	3.20E+01	3.83E-12
	SD	2.49E+01	1.87E+02	1.61E+02	2.15E+01	2.28E+01	1.83E-12
f9	Min	1.00E+02	5.02E+01	2.09E+02	2.21E+02	2.13E+02	2.33E-13
	Max	8.24E+02	3.71E+02	3.78E+02	6.14E+02	6.15E+02	7.12E-13
	Median	5.46E+02	2.78E+02	2.77E+02	3.50E+02	3.22E+02	4.62E-13
	SD	1.67E+02	6.54E+01	4.83E+01	8.97E+01	8.30E+01	1.18E-13
f10	Min	2.85E+01	3.47E+01	3.46E+01	3.15E+01	3.30E+01	1.24E+01
	Max	1.05E+02	1.03E+02	9.99E+01	7.15E+01	9.51E+01	1.99E+01
	Median	6.07E+01	7.20E+01	6.65E+01	5.56E+01	6.53E+01	1.84E+01
	SD	1.87E+01	2.01E+01	1.75E+01	9.95E+00	1.69E+01	1.20E+00
f11	Min	-1.47E+01	-2.69E+01	-5.84E+01	-6.03E+01	-9.32E+01	-1.03E+02
	Max	3.96E+01	1.96E+01	2.03E+01	4.17E+01	4.28E+01	1.38E+01
	Median	-1.54E+01	-1.06E+01	-0.71E+01	-2.57E+01	-3.43E+01	-1.87E+01
	SD	9.22E+00	1.54E+01	2.26E+01	3.24E+01	3.52E+01	3.83E+01
f12	Min	5.09E+02	6.01E+03	3.27E+01	6.04E+01	3.21E+01	1.24E-02
	Max	1.45E+04	7.27E+04	2.01E+03	2.00E+02	6.80E+01	9.58E-02
	Median	2.37E+03	6.72E+03	6.59E+01	1.41E+02	4.05E+01	2.90E-02
	SD	1.74E+03	1.88E+03	2.02E+02	3.46E+01	9.32E+00	2.74E-02
f13	Min	3.17E+03	6.54E+04	2.76E+03	4.16E+03	1.15E+06	1.51E+04
	Max	1.06E+05	5.38E+05	1.10E+05	1.47E+05	1.69E+07	2.56E+05
	Median	5.08E+04	9.03E+04	7.08E+04	6.32E+04	6.48E+06	4.23E+04
	SD	3.02E+04	1.44E+05	4.68E+04	4.70E+04	5.48E+06	8.38E+04
f14	Min	4.42E+05	8.92E+05	7.10E+04	2.35E+05	7.90E+07	7.29E+06
	Max	3.60E+07	1.55E+07	7.20E+08	6.37E+07	3.61E+09	7.83E+07
	Median	1.99E+07	1.18E+07	1.16E+07	1.01E+07	9.31E+08	1.01E+07
	SD	1.79E+07	5.69E+06	3.45E+08	2.82E+07	1.27E+09	2.76E+07
f15	Min	3.44E+03	3.78E+03	2.94E+03	3.49E+03	6.21E+04	1.97E+03
	Max	1.07E+06	5.79E+03	5.01E+03	4.58E+03	5.62E+06	5.75E+04
	Median	4.11E+03	4.60E+03	3.93E+03	3.99E+03	4.85E+05	5.75E+03
	SD	3.37E+05	5.61E+02	6.06E+02	3.54E+02	1.70E+06	1.80E+05
f16	Min	2.64E+03	2.66E+03	2.64E+03	2.64E+03	6.02E+03	2.50E+03
	Max	3.09E+03	2.84E+03	2.79E+03	3.15E+03	8.40E+03	2.50E+03
	Median	2.70E+03	2.79E+03	2.65E+03	2.75E+03	6.99E+03	2.50E+03
	SD	1.37E+02	7.80E+01	5.52E+01	1.54E+02	8.94E+02	3.30E-04
f17	Min	2.74E+03	2.92E+03	2.73E+03	2.74E+03	3.16E+03	2.60E+03
	Max	2.85E+03	3.11E+03	3.05E+03	2.98E+03	3.39E+03	2.60E+03
	Median	2.78E+03	3.02E+03	2.80E+03	3.32E+03	3.32E+03	2.60E+03
	SD	3.27E+01	6.75E+01	1.04E+02	8.61E+01	5.98E+01	3.97E-03
f18	Min	2.72E+03	2.76E+03	2.74E+03	2.76E+03	2.91E+03	2.70E+03
	Max	2.78E+03	2.84E+03	2.80E+03	2.81E+03	3.33E+03	2.70E+03
	Median	2.76E+03	2.78E+03	2.77E+03	2.77E+03	3.16E+03	2.70E+03
	SD	1.93E+01	2.15E+01	1.96E+01	1.98E+01	1.22E+02	1.75E-05

Table 6. Comparisons between the AIWPSO and the other PSOs on Wilcoxon tests based on 30-dimensional benchmark functions ($f_1 - f_{18}$). (*, $P < 0.05$, the difference is significant; 0 , $P > 0.05$, the difference is not significant.)

	CPSO	RPSO	LDPSO	NLDPSO	APSO
f1	3.0161E-11*	3.0142E-11*	3.0161E-11*	3.0047E-11*	3.0142E-11*
f2	3.0123E-11*	2.2521E-04*	2.3486E-05*	6.0003E-08*	2.9841E-11*
f3	3.9947E-11*	2.9822E-11*	2.9785E-11*	8.3853E-11*	5.9560E-11*
f4	3.0010E-11*	3.0029E-11*	1.0242E-06*	3.9789E-10*	7.4067E-09*
f5	2.9654E-11*	2.9580E-11*	2.9673E-11*	2.9691E-11*	2.9673E-11*
f6	2.9450E-11*	2.9524E-11*	2.9284E-11*	2.9487E-11*	2.9192E-11*
f7	3.0199E-11*	3.0180E-11*	3.0180E-11*	3.0199E-11*	3.0104E-11*
f8	2.8754E-11*	2.8954E-11*	2.8718E-11*	2.8791E-11*	2.8520E-11*
f9	2.9803E-11*	2.9916E-11*	2.9935E-11*	2.9897E-11*	2.9935E-11*
f10	2.9027E-11*	2.9009E-11*	2.9027E-11*	2.8845E-11*	2.8991E-11*
f11	2.3824E-05*	3.6214E-05*	2.0020E-05*	2.9655E-04*	4.1032E-03*
f12	3.0148E-11*	3.0080E-11*	3.0080E-11*	3.0291E-11*	3.0114E-11*
f13	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	2.2022E-03*
f14	3.8467E-01 ⁰	3.7635E-02*	1.4047E-01 ⁰	6.4022E-02 ⁰	1.0411E-01 ⁰
f15	1.6197E-01 ⁰	3.7635E-02*	1.4047E-01 ⁰	3.7635E-02*	2.1134E-02*
f16	1.8267E-04*	4.3964E-04*	1.8267E-04*	4.3964E-04*	1.0411E-02*
f17	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*
f18	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*

Table 7. Comparisons between the AIWPSO and the other PSOs on Wilcoxon tests based on 40-dimensional benchmark functions ($f_1 - f_{18}$). (*, $P < 0.05$, the difference is significant; 0 , $P > 0.05$, the difference is not significant.)

	CPSO	RPSO	LDPSO	NLDPSO	APSO
f1	3.0085E-11*	3.0104E-11*	2.9878E-11*	3.0047E-11*	2.9747E-11*
f2	3.0047E-11*	5.1819E-07*	8.2800E-06*	7.3524E-10*	1.0550E-09*
f3	2.9878E-11*	2.9972E-11*	6.9579E-11*	2.9953E-11*	2.7689E-09*
f4	3.8074E-10*	3.0085E-11*	3.7968E-07*	3.8053E-10*	5.0820E-06*
f5	3.0066E-11*	3.0123E-11*	3.0161E-11*	3.0261E-11*	3.0123E-11*
f6	2.9935E-11*	3.0142E-11*	3.0085E-11*	3.0123E-11*	3.0142E-11*
f7	3.0119E-11*	3.0161E-11*	3.0199E-11*	3.0142E-11*	3.0180E-11*
f8	3.0123E-11*	3.0066E-11*	3.0323E-11*	3.0143E-11*	3.0104E-11*
f9	2.0123E-11*	3.0142E-11*	3.0123E-11*	3.0085E-11*	3.0142E-11*
f10	2.9636E-11*	2.9616E-11*	2.9617E-11*	2.9598E-11*	2.9636E-11*
f11	3.1113E-05*	2.9152E-04*	3.4141E-04*	3.0896E-03*	2.8426E-03*
f12	3.0019E-11*	3.0083E-11*	3.0094E-11*	3.0043E-11*	3.0062E-11*
f13	6.2186E-01 ⁰	8.7772E-02 ⁰	7.3266E-01 ⁰	9.6974E-01 ⁰	1.7265E-04*
f14	4.6744E-01 ⁰	3.0346E-01 ⁰	6.7516E-01 ⁰	6.7516E-01 ⁰	1.6305E-04*
f15	2.4613E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	3.6105E-03*
f16	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*
f17	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*
f18	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*	1.8267E-04*

Table 8. Comparisons and analysis of the search results of the different algorithms based on 30 dimensional benchmark functions ($f_1 - f_4$).

		Alireza	Fernandes
f1	Min	5.23E-01	1.09E-03
	Max	7.43E+00	1.80E-02
	Median	4.01E+00	3.27E-03
	SD	2.03E+00	2.78E-03
f2	Min	1.38E-01	5.20E-01
	Max	2.33E+01	1.99E+02
	Median	9.79E-01	9.91E-01
	SD	4.10E+00	3.23E+01
f3	Min	2.72E+03	1.23E+01
	Max	6.43E+04	1.94E+02
	Median	2.71E+04	2.46E+01
	SD	1.92E+04	4.01E+01
f4	Min	1.01E-01	1.00E-02
	Max	2.11E+03	5.33E+02
	Median	1.02E+01	8.31E-02
	SD	3.21E+03	2.92E+03

Table 9. Computational cost comparisons based on 30 dimensional benchmark functions ($f_1 - f_{18}$).

Function	CPSO	RPSO	LDPSO	NLDPSO	APSO	AIWPSO
f1 Mean iterations	7106	-	4312	4426	5213	2568
Times(s)	1.7765	-	1.078	1.1065	1.30325	0.642
f2 Mean iterations	3023	19210	5551	4104	3086	1952
Times(s)	0.75575	4.8025	1.38775	1.026	0.7715	0.488
f3 Mean iterations	688	-	2439	2522	10632	1722
Times(s)	0.172	-	0.60975	0.6305	2.658	0.4305
f4 Mean iterations	3106	-	4843	2402	3376	1965
Times(s)	0.7765	-	1.21075	0.6005	0.844	0.49125
f5 Mean iterations	-	8934	8687	6433	8349	1885
Times(s)	-	2.2335	2.17175	1.60825	2.08725	0.47125
f6 Mean iterations	648	1127	582	553	631	1621
Times(s)	0.162	0.28175	0.1455	0.13825	0.15775	0.40525
f7 Mean iterations	10615	-	2366	2573	2054	2969
Times(s)	2.65375	-	0.5915	0.64325	0.5135	0.74225
f8 Mean iterations	1263	18	14	65	21	12
Times(s)	0.31575	0.0045	0.0035	0.01625	0.00525	0.003
f9 Mean iterations	-	-	-	-	-	3706
Times(s)	-	-	-	-	-	22.236
f10 Mean iterations	141	99	139	122	91	144
Times(s)	0.7896	0.5544	0.7784	0.6832	0.5096	0.8064
f11 Mean iterations	13284	9799	9021	6531	7128	6942
Times(s)	74.0583	54.6294	50.2921	36.4103	39.7386	38.7016
f12 Mean iterations	14565	-	5098	6043	4182	6206
Times(s)	3.6412	-	1.2745	1.5107	1.0455	1.5515
f13 Mean iterations	1901	9019	10176	7249	-	13281
Times(s)	7.4137	7.5781	8.2582	8.4453	-	13.8265
f14 Mean iterations	16345	14141	21055	8752	-	15930
Times(s)	7.8179	7.9781	8.5468	8.8599	-	14.8335
f15 Mean iterations	30641	16139	14417	7757	-	14520
Times(s)	8.1356	8.1768	8.865	9.0919	-	15.7617
f16 Mean iterations	16325	1683	6145	2366	245	7815
Times(s)	13.1355	13.1964	13.8409	13.9815	14.5508	25.2475
f17 Mean iterations	15852	1528	6433	1887	330	7037
Times(s)	11.0261	11.0371	11.7644	11.8926	12.3177	21.2807
f18 Mean iterations	16142	1150	6185	2086	1294	7604
Times(s)	11.8693	11.896	12.662	12.7903	13.2452	23.0088